Programowanie obiektowe i inżynieria oprogramowania

UML 4:

- Diagram maszyny stanu;
- Testowanie modelu.

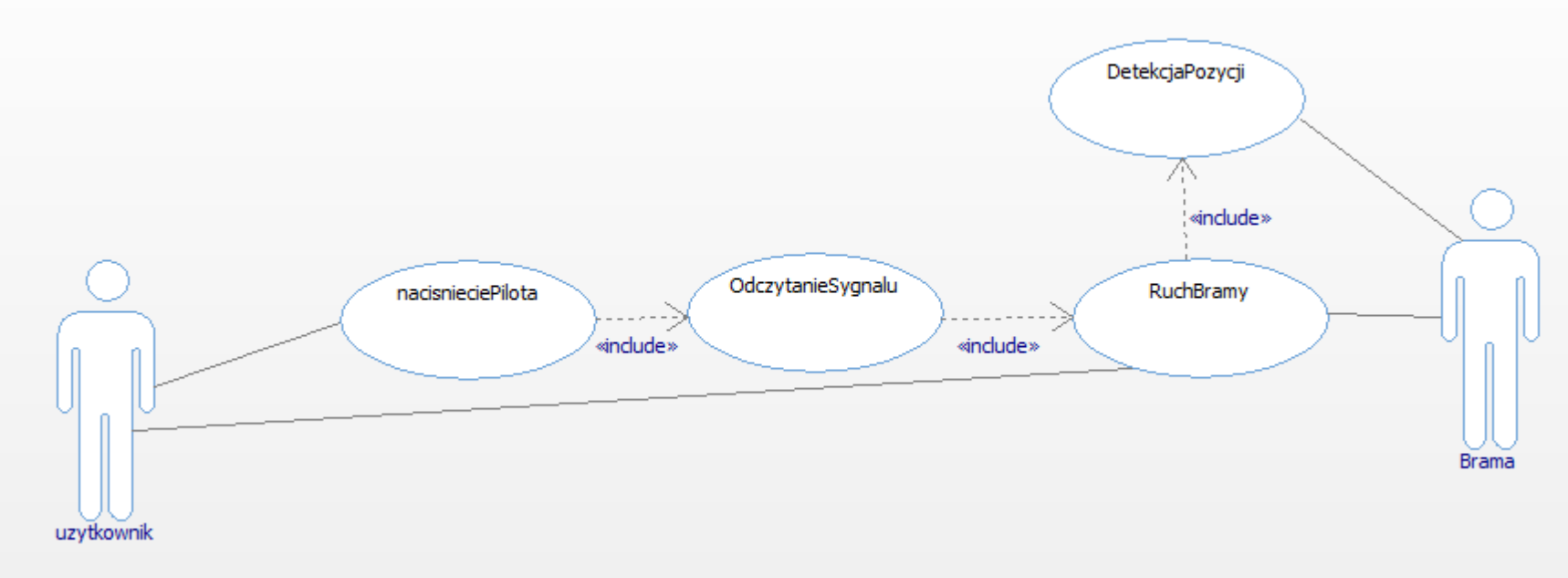
Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: mgr inż. Jakub Bryła
2023

Najprostszy przykład

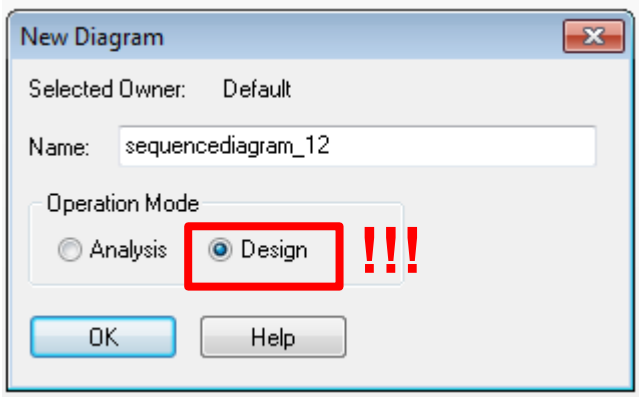
Opis przykładu

Diagram przypadków użycia



Prezentowany przykład prezentuje możliwość sterowania bramą przez pilota lub bezpośrednio przez przyciski na bramie – stąd dwie strzałki asocjacji od użytkownika.

Diagram sekwencji



Opis przykładu

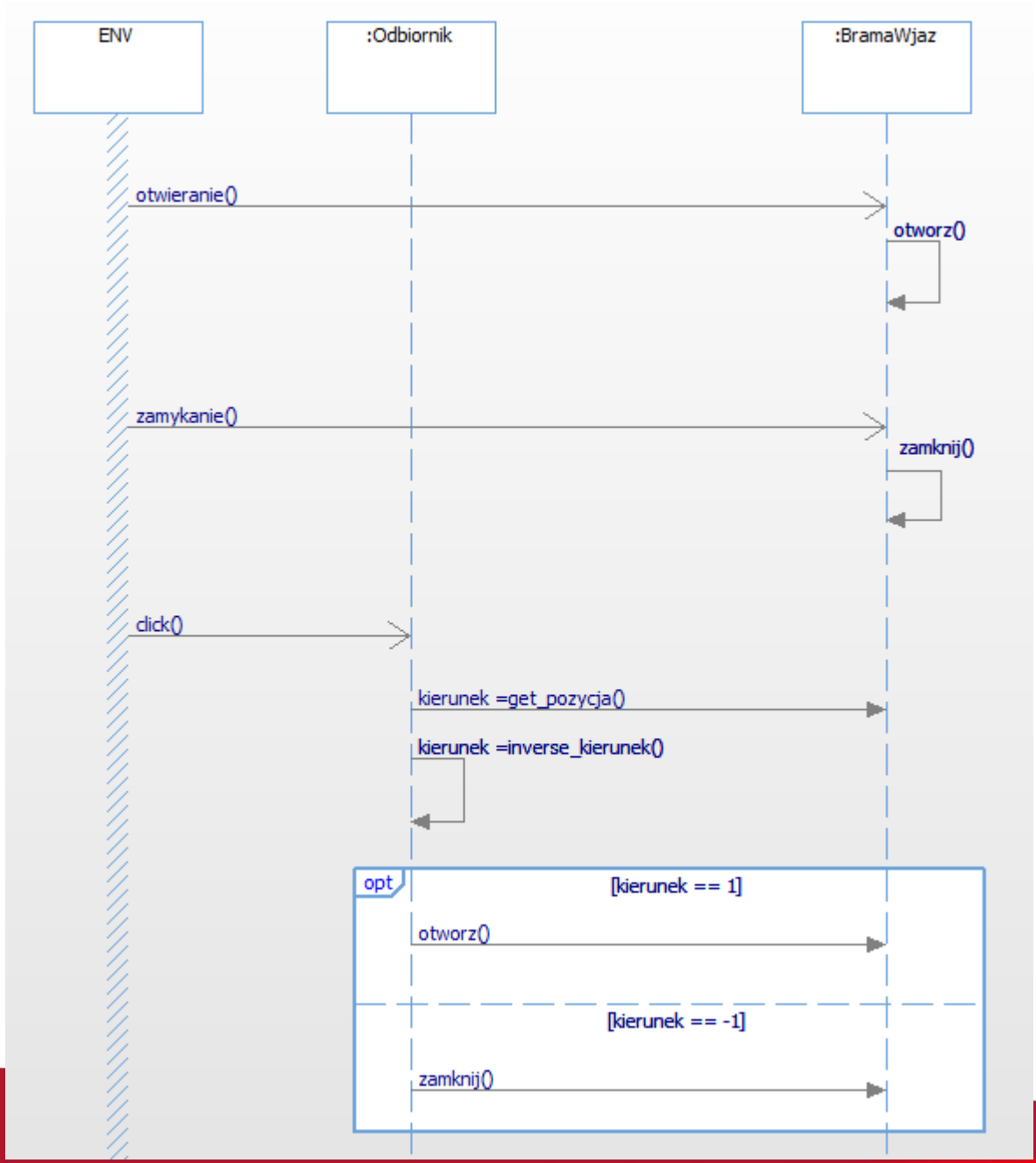
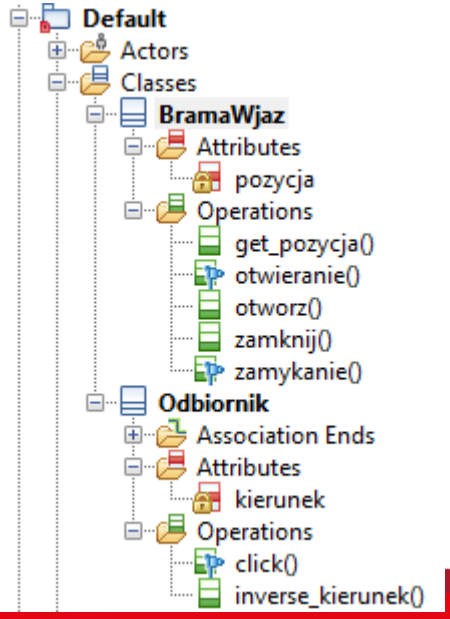


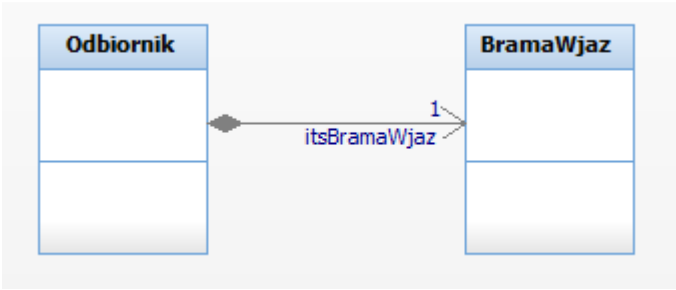
Diagram sekwencji

Zakładamy, że brama posiada dwa niezależne przyciski do otwierania / zamykania. Pilot za każdym razem pyta bramę w jakiej jest pozycji.
Pilot posiada atrybut kierunek, brama zmienną składową pozycja. Obie zainicjalizowane wartością -1.



Opis przykładu

Diagram klas



Mamy tylko dwie klasy, z czego tylko odbiornik potrzebuje referencji do bramy – do uzyskania informacji o aktualnej pozycji.

Najprostszy przykład

Diagram maszyny stanu

Diagram maszyny stanu

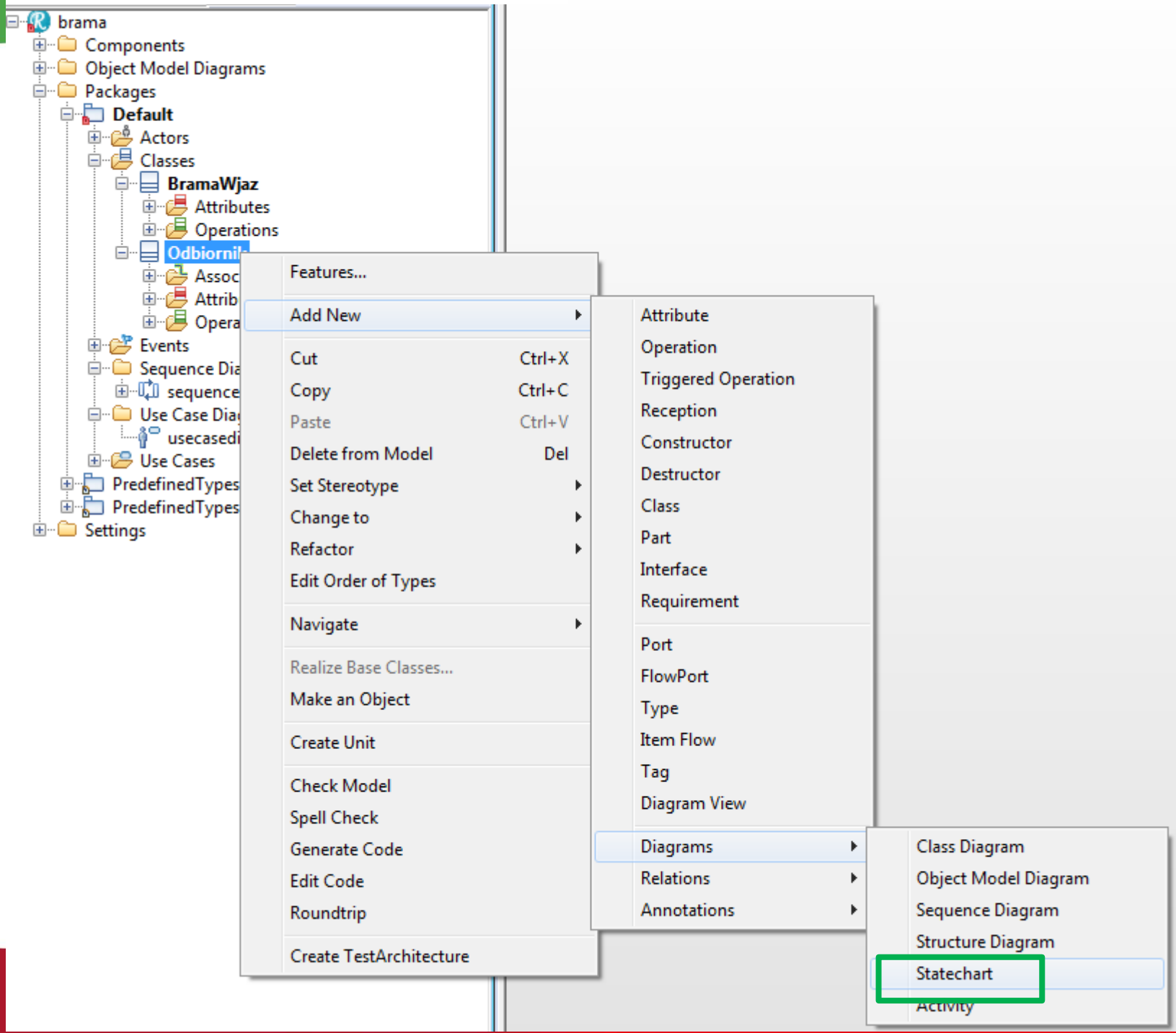
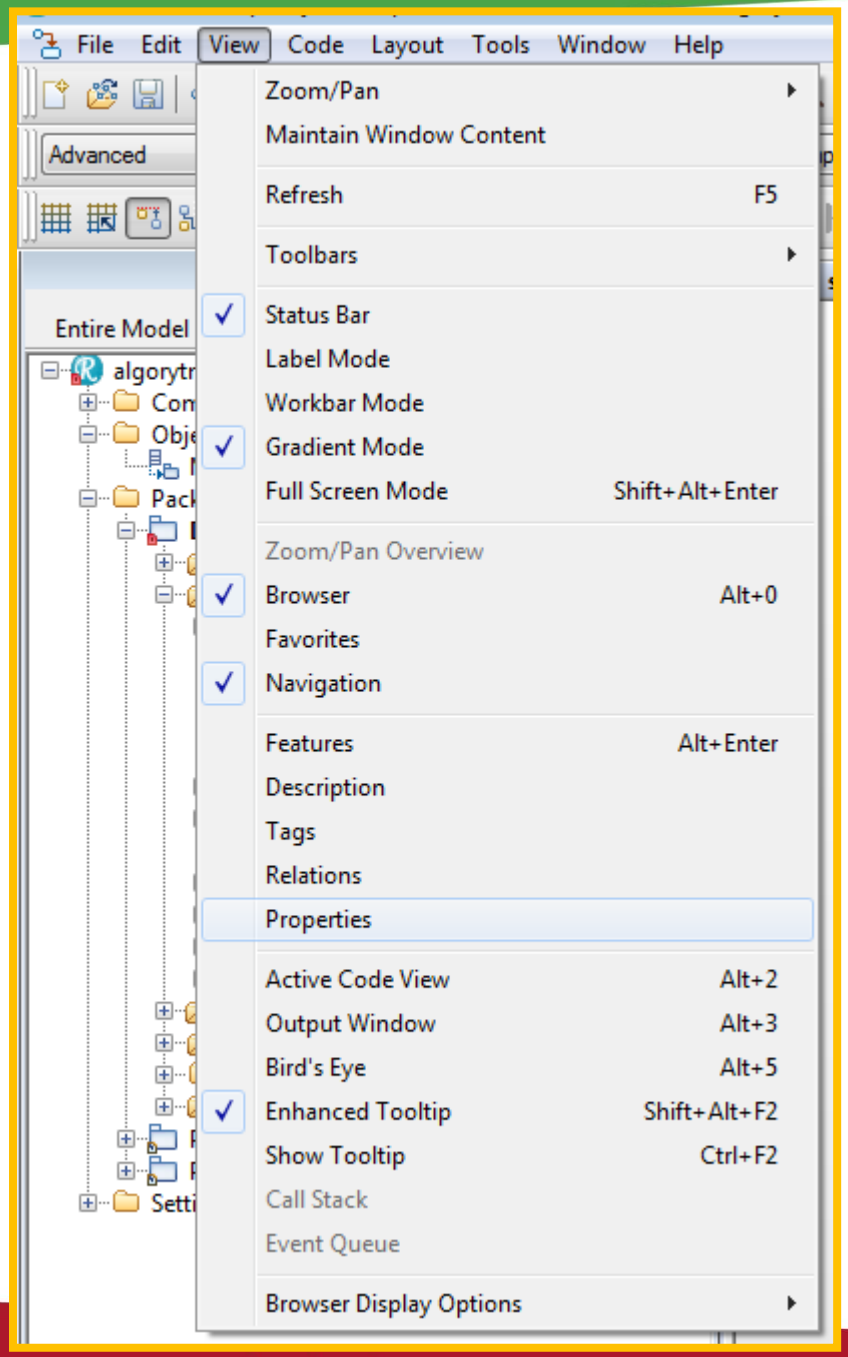


Diagram maszyny stanu



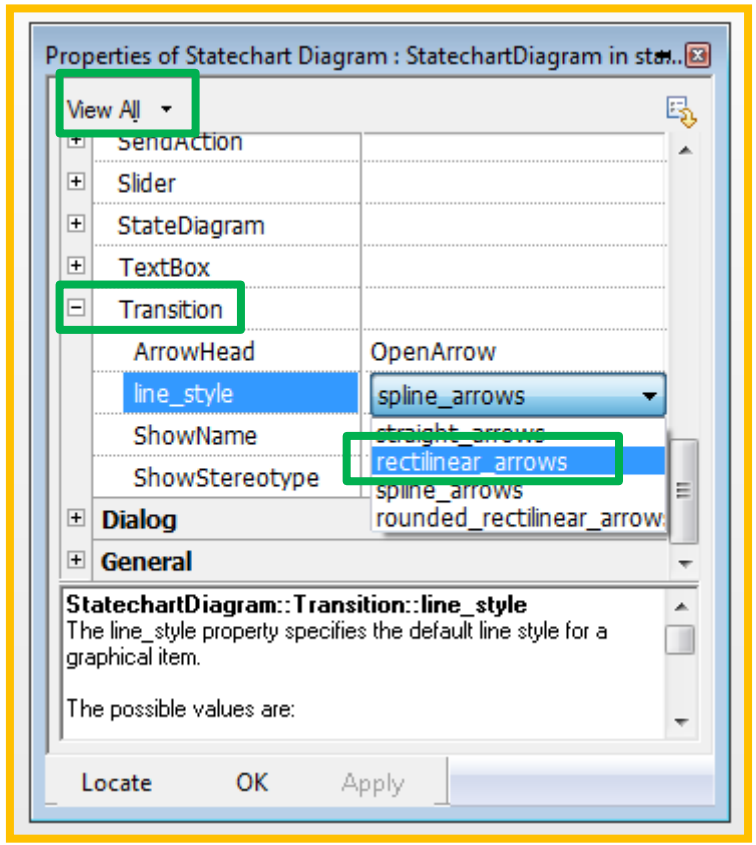
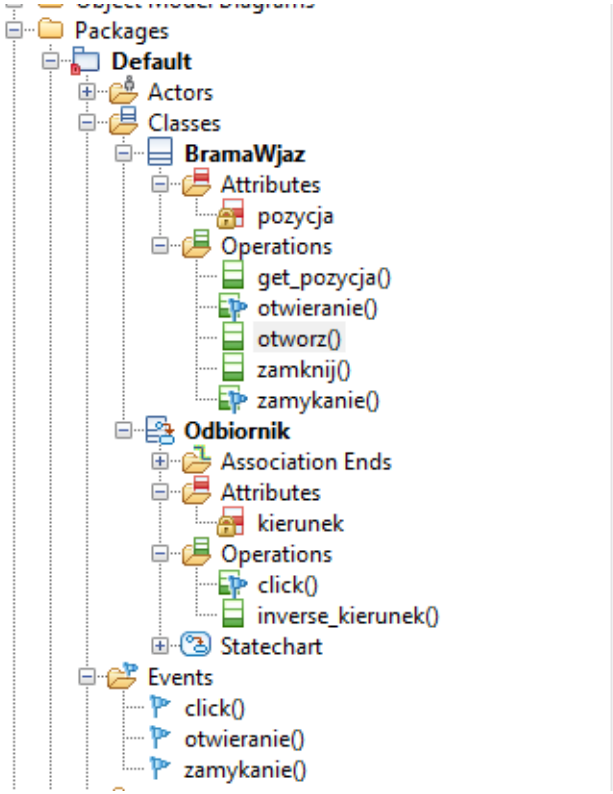
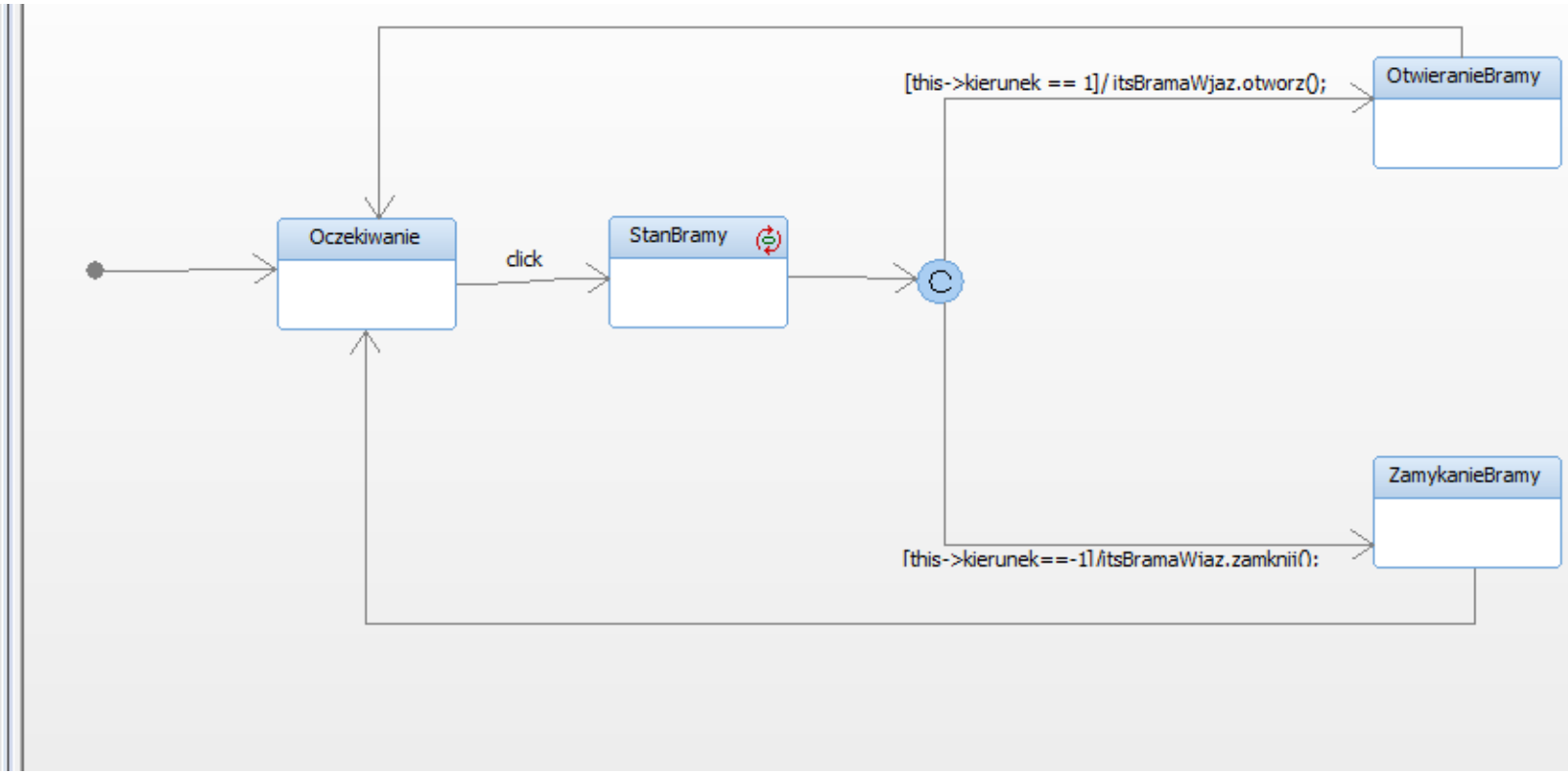


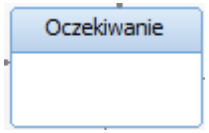
Diagram maszyny stanu



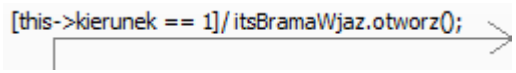


Stan początkowy

Warunek if

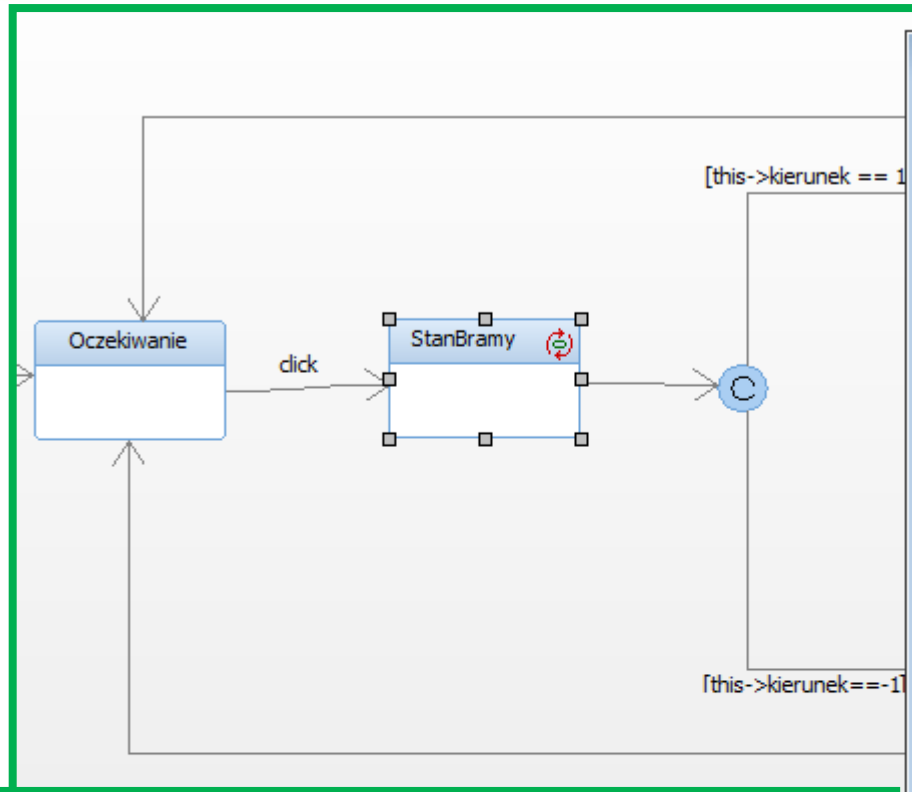


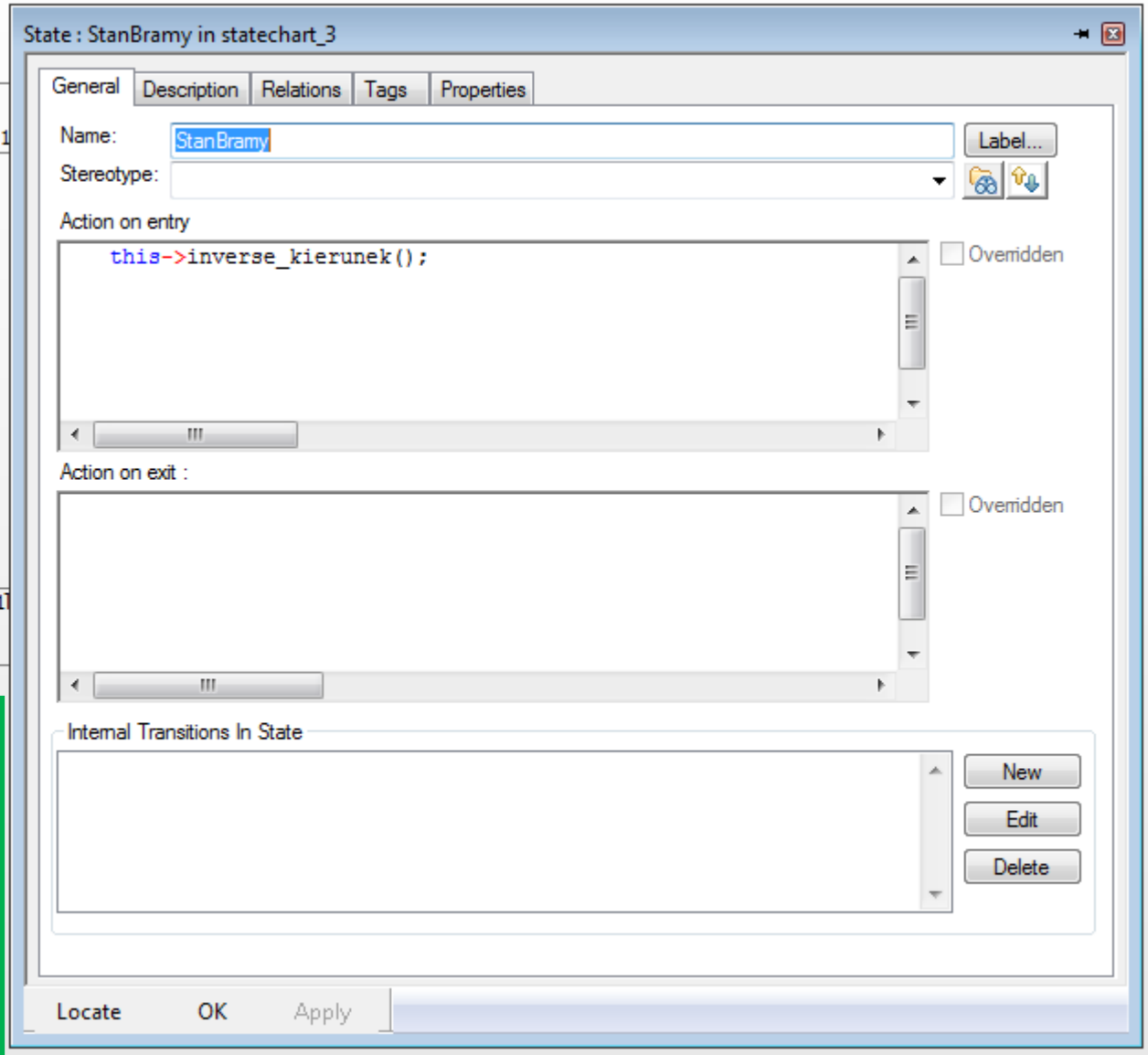
Stan klasy



Przejście – może być automatyczne, wyzwalane zdarzeniem, z [warunkiem zajścia], z wywoływanym kodem

Diagram maszyny stanu





Domyślnie odbiornik jest w stanie oczekiwania na zajście zdarzenia *clik* – naciśnięcie przycisku pilota. Wtedy przechodzi do stanu StanBramy, w którym wywoływana jest funkcja odbiornika *inverse_kierunek()*.

Diagram maszyny stanu

Operation : inverse_kierunek in Odbiornik

General | Description | Implementation | Arguments | Relations | Tags | Properties

void inverse_kierunek()

```
int pozycja = itsBramaWjaz.get_pozycja();

if (pozycja == -1) this->kierunek=1;
else this->kierunek=-1;
```

Locate OK Apply

Implementacja funkcji inverse_kierunek().

Do zmiennych/funkcji składowych klasy, dla której tworzony jest diagram stanu, odwołujemy się za pomocą this->….

Operation : get_pozycja in BramaWjaz

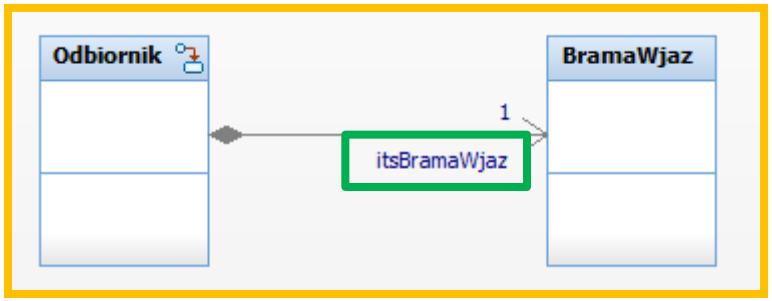
General | Description | Implementation | Arguments | Relations | Tags | Properties

int get_pozycja()

```
return this->pozycja;
```

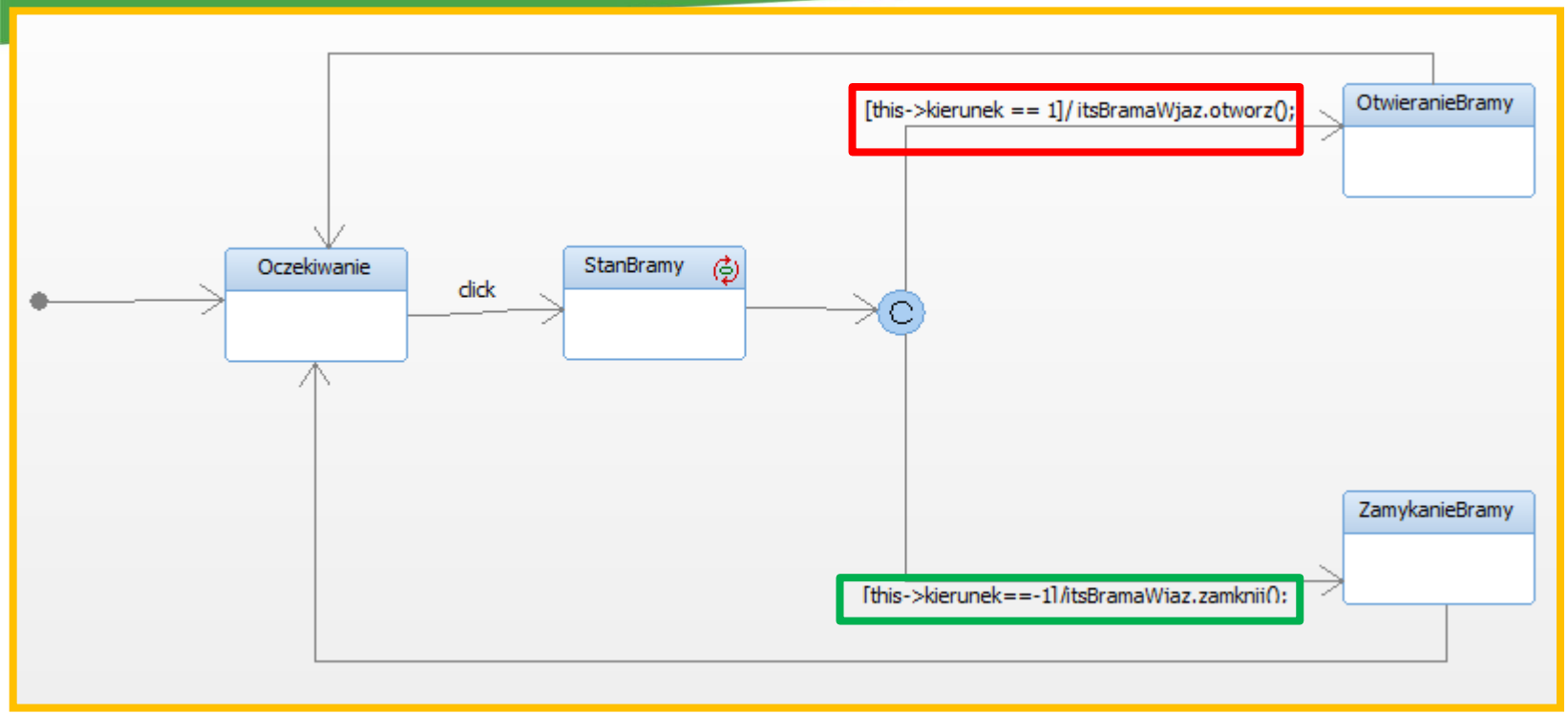
Locate OK Apply

Implementacja funkcji *get_pozycja().*

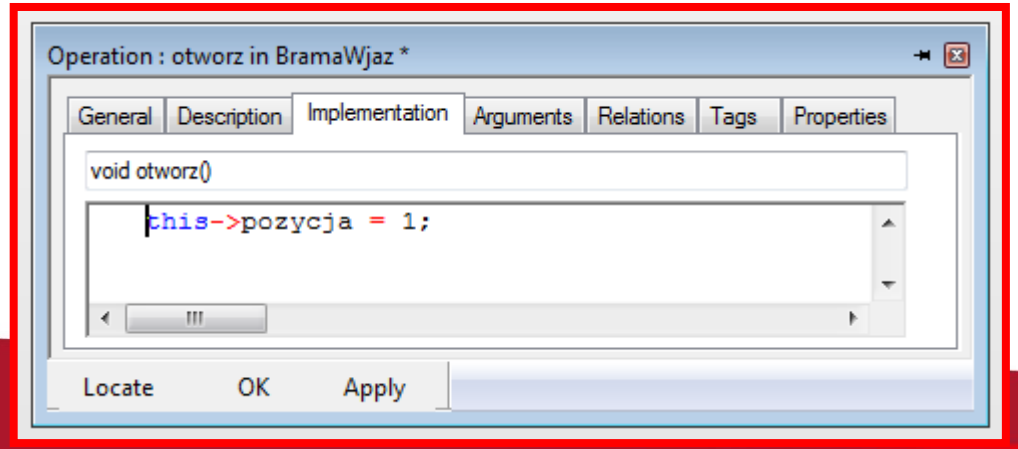


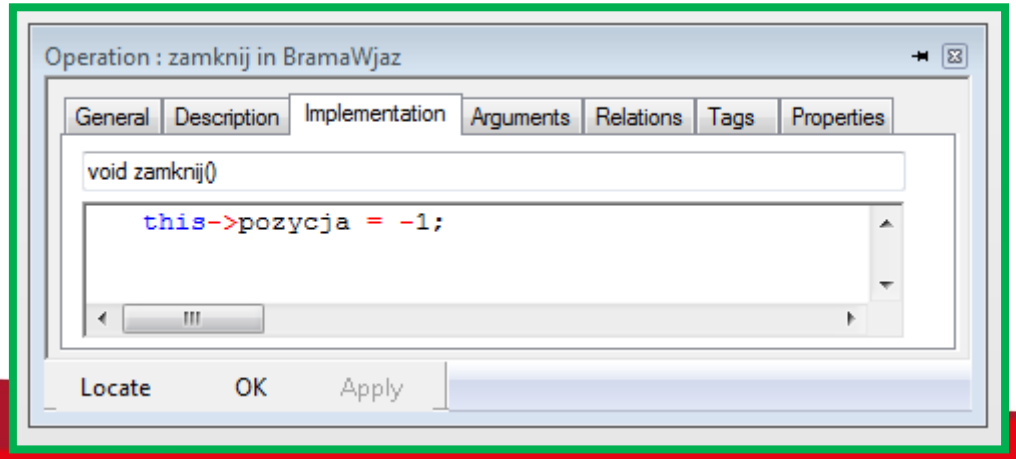
Do zmiennych/funkcji składowych klas odwołujemy się za pomocą nazwy z diagramu klas.

Diagram maszyny stanu



[this->kierunek == 1] / itsBramaWjaz.otworz(); // jeśli parametr składowy *kierunek* klasy *Odbiornik* przyjmie wartość 1 zostanie wykonana funkcja składowa *otworz()* obiektu *itsBramaWjaz* (klasy *BramaWjaz*).





Podsumowanie

Diagram przypadków użycia – określenie celów, przeznaczenia programu;

Diagram sekwencji – określenie przepływu informacji pomiędzy uczestnikami; identyfikacja potrzebnych klas, ich parametrów i funkcji składowych;

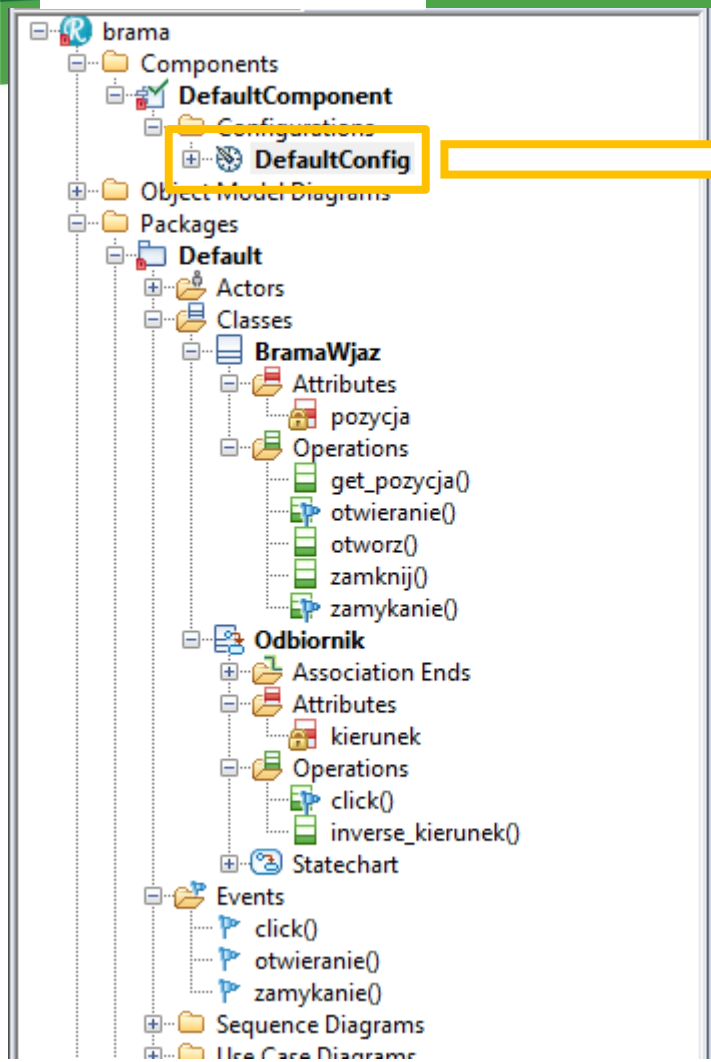
Diagram klas – określenie zależności między klasami;

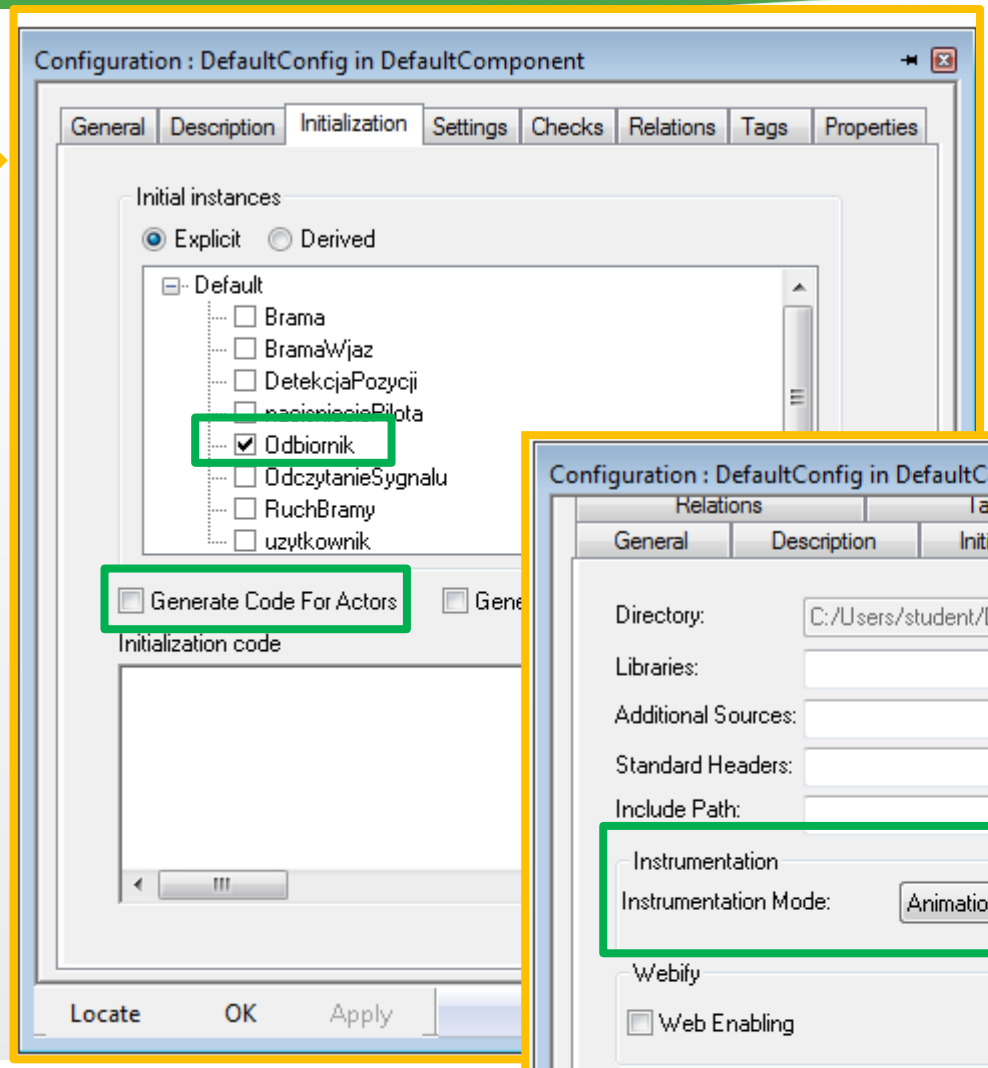
Diagram maszyny stanów – określenie zmian stanu klas na podstawie zdarzeń zidentyfikowanych na diagramie sekwencji. Implementacja kodu od strony przepływu sterowania.

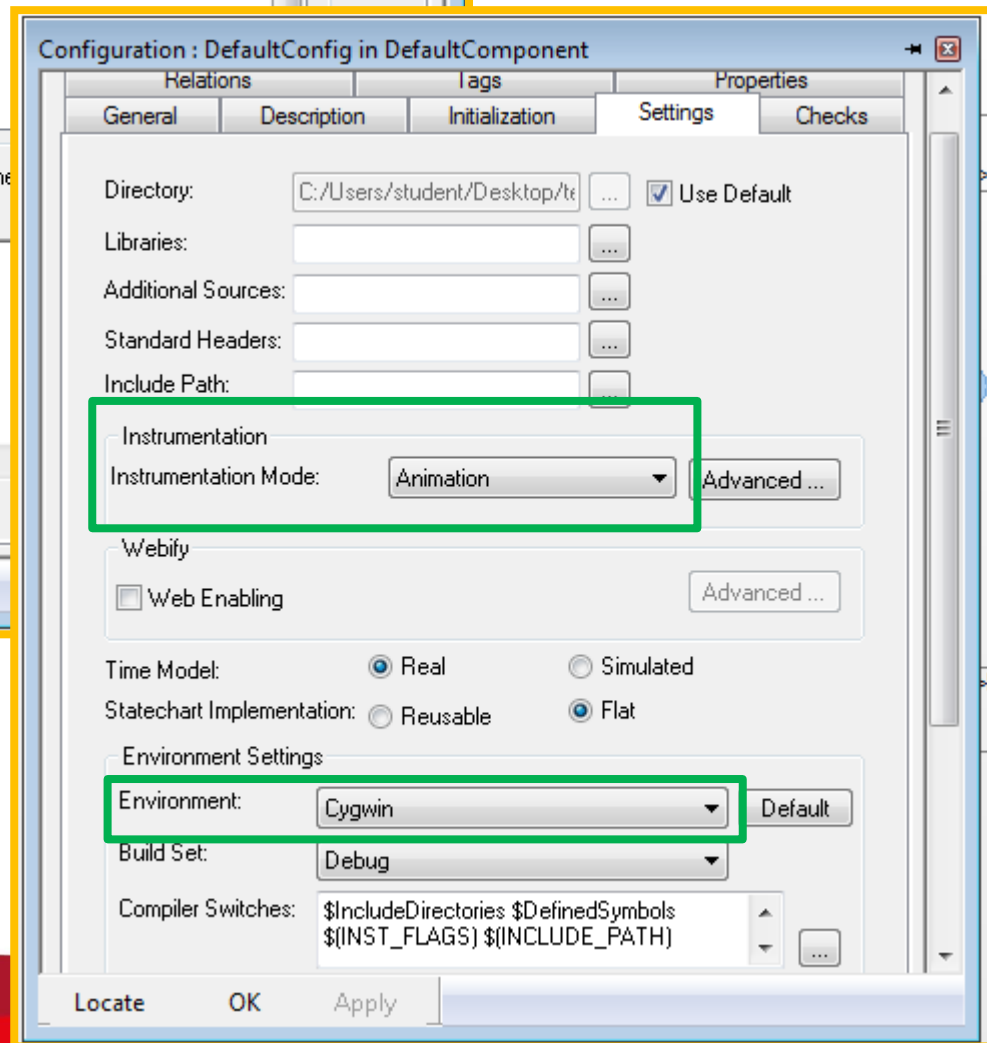
Najprostszy przykład

Testowanie

Testowanie


brama
Components
DefaultComponent
DefaultConfig
Packages
Default
Actors
Classes
BramaWjaz
Attributes
pozycja
Operations
get_pozycja()
otwieranie()
otworz()
zamknij()
zamykanie()
Odbiornik
Association Ends
Attributes
kierunek
Operations
click()
inverse_kierunek()
Statechart
Events
click()
otwieranie()
zamykanie()
Sequence Diagrams

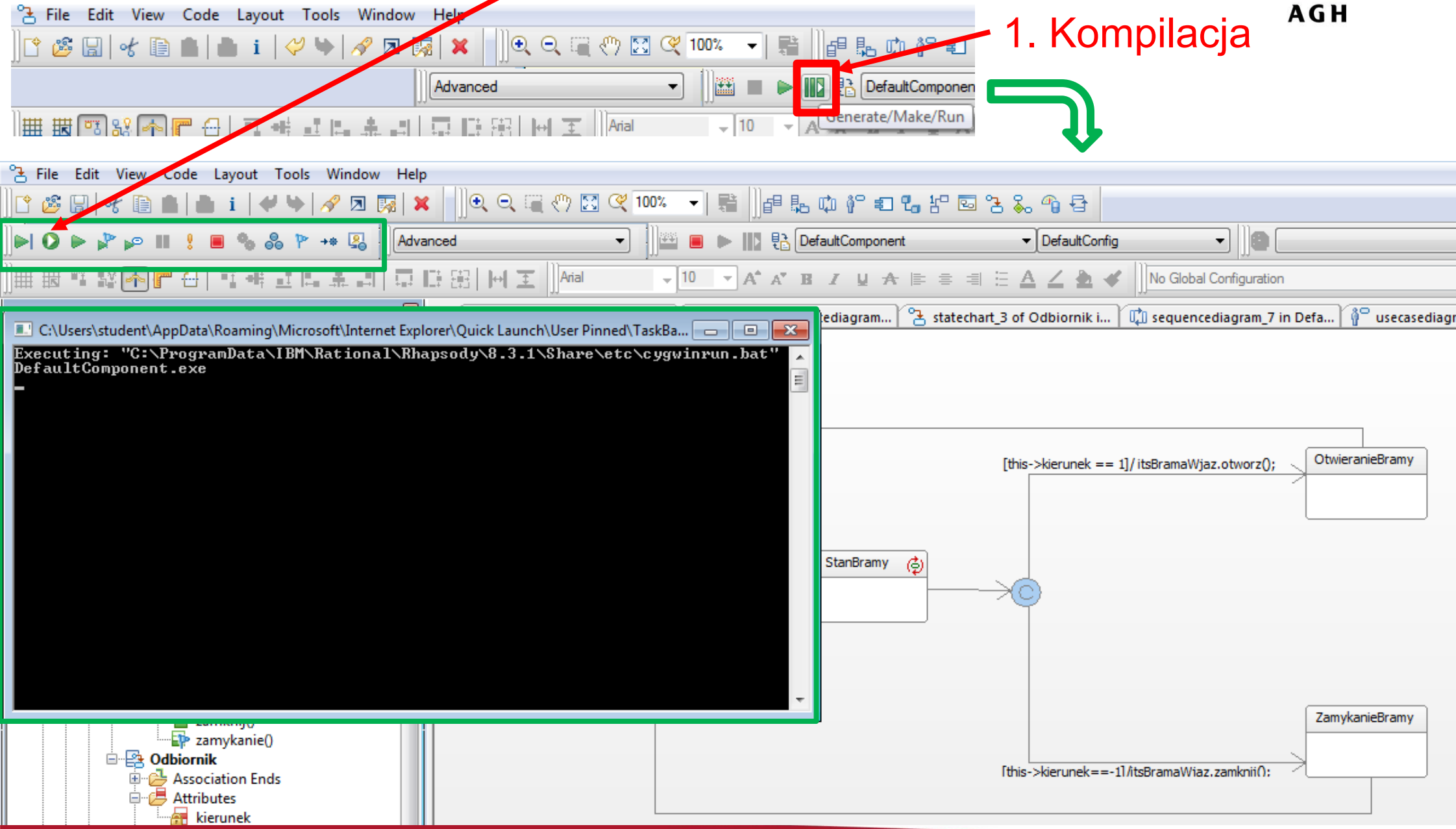

Configuration : DefaultConfig in DefaultComponent
General
Description
Initialization
Settings
Checks
Relations
Tags
Properties
Initial instances
Explicit
Derived
Default
Brama
BramaWjaz
DetekcjaPozycji
Odbiornik
OdczytanieSygnalu
RuchBramy
uzytkownik
Generate Code For Actors
Initialization code
Locate
OK
Apply


Configuration : DefaultConfig in DefaultComponent
Relations
Tags
Properties
General
Description
Initialization
Settings
Checks
Directory:
C:/Users/student/Desktop/te
Use Default
Libraries:
Additional Sources:
Standard Headers:
Include Path:
Instrumentation
Instrumentation Mode:
Animation
Advanced ...
Webify
Web Enabling
Advanced ...
Time Model:
Real
Simulated
Statechart Implementation:
Reusable
Flat
Environment Settings
Environment:
Cygwin
Default
Build Set:
Debug
Compiler Switches:
$IncludeDirectories $DefinedSymbols
$(INST_FLAGS) $(INCLUDE_PATH)
Locate
OK
Apply

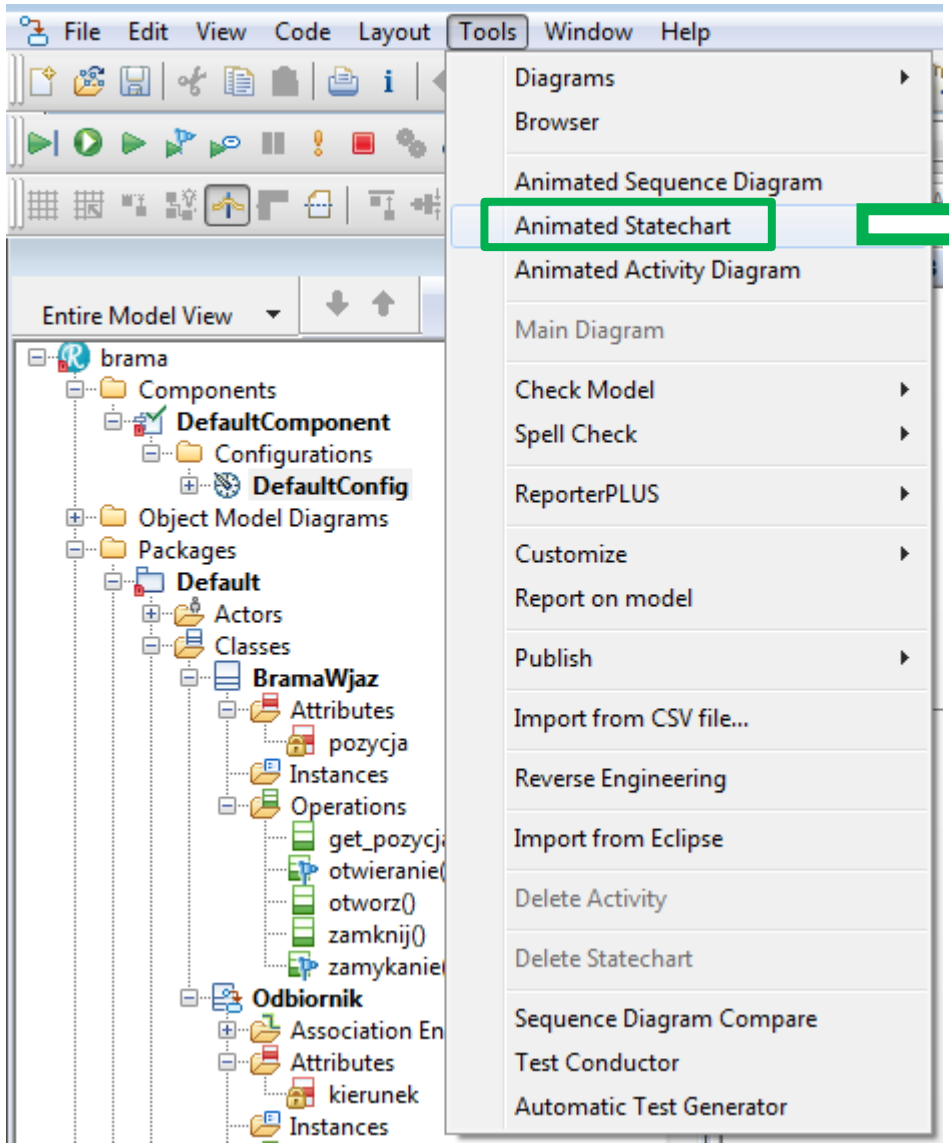
Testowanie

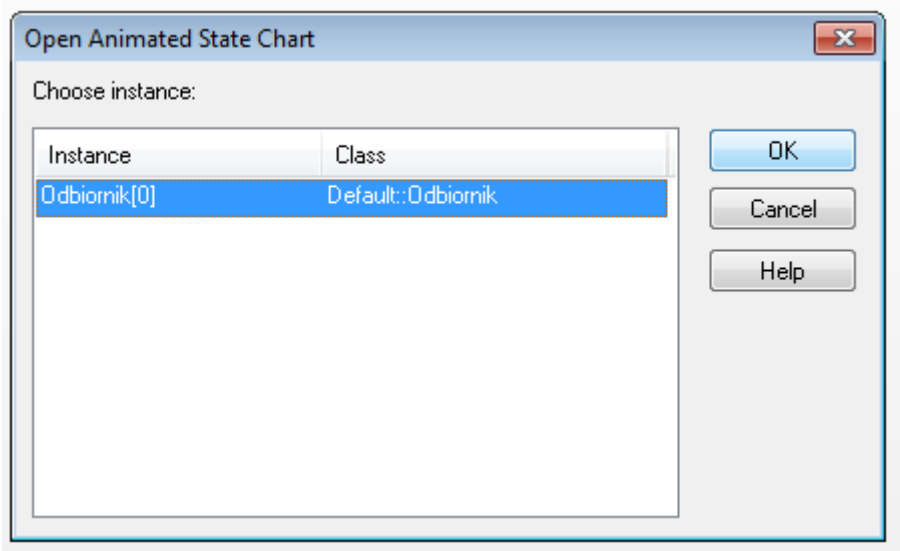
2. Uruchomienie programu

1. Kompilacja



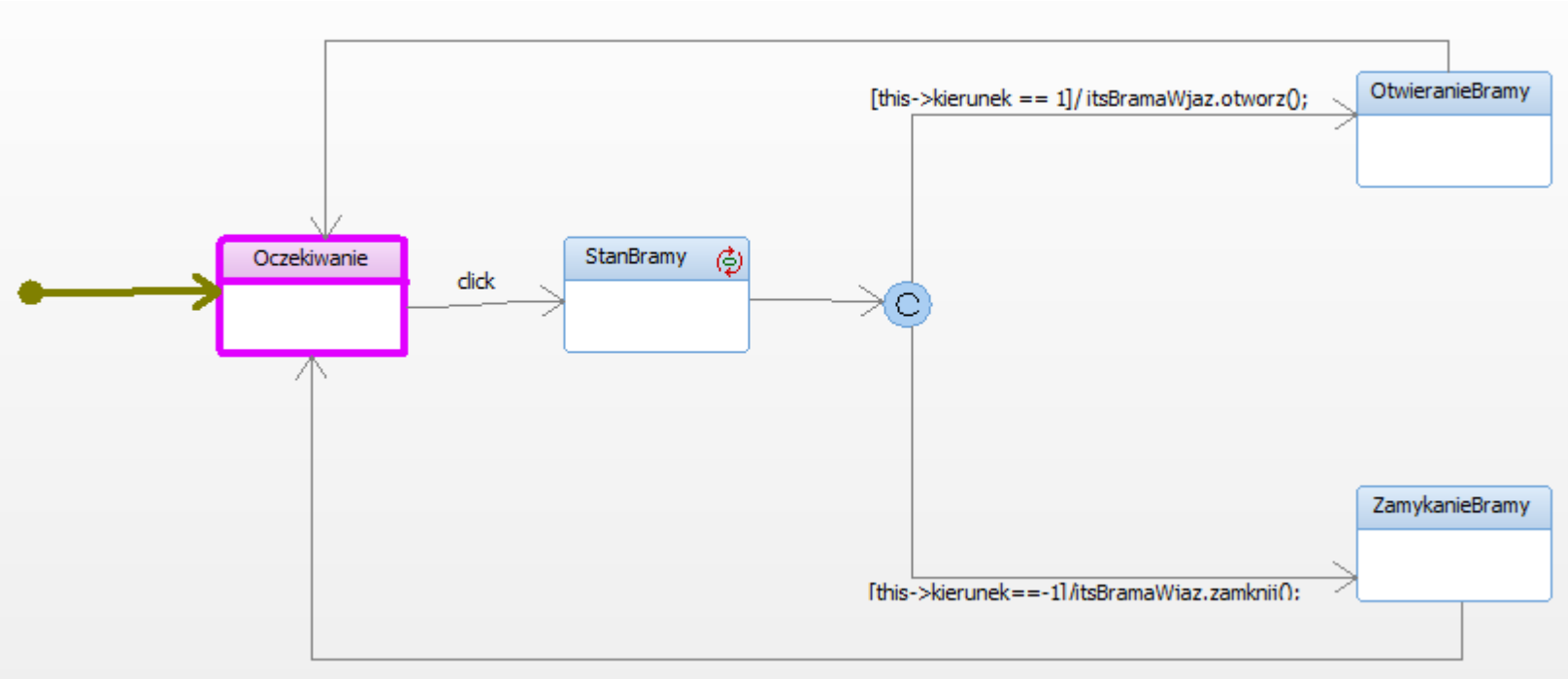
Testowanie





Śledzenie zmian na wybranym diagramie maszyny stanu w trakcie testowania.

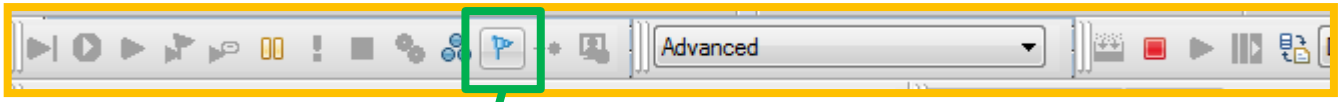
Testowanie

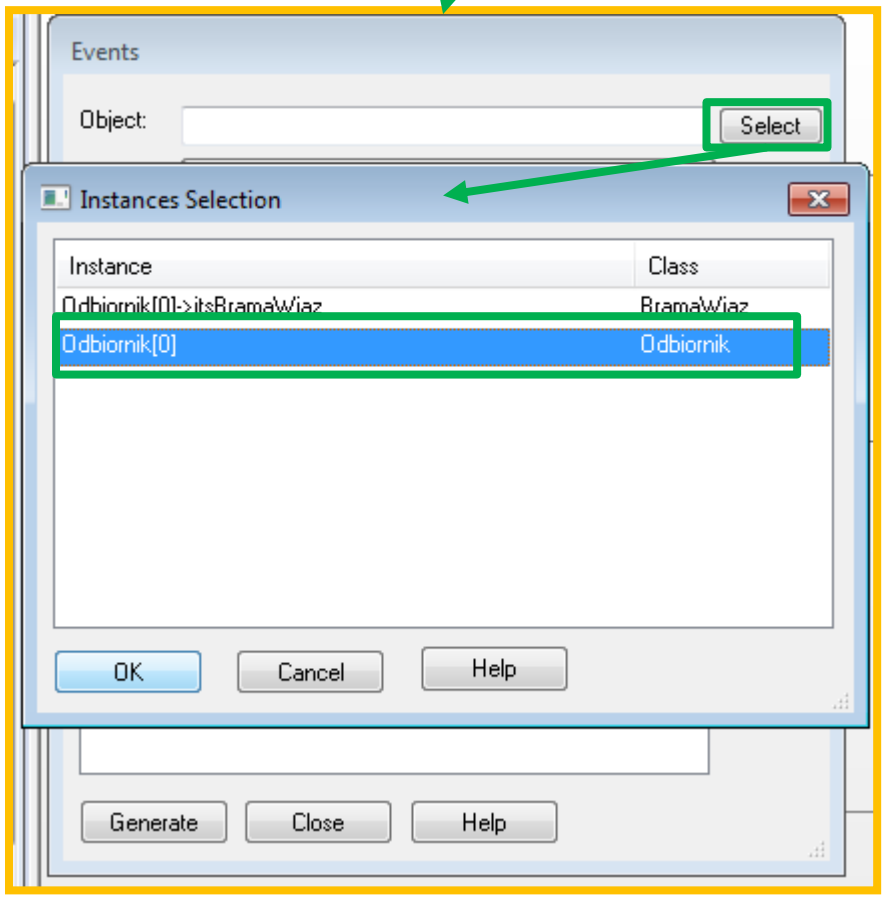


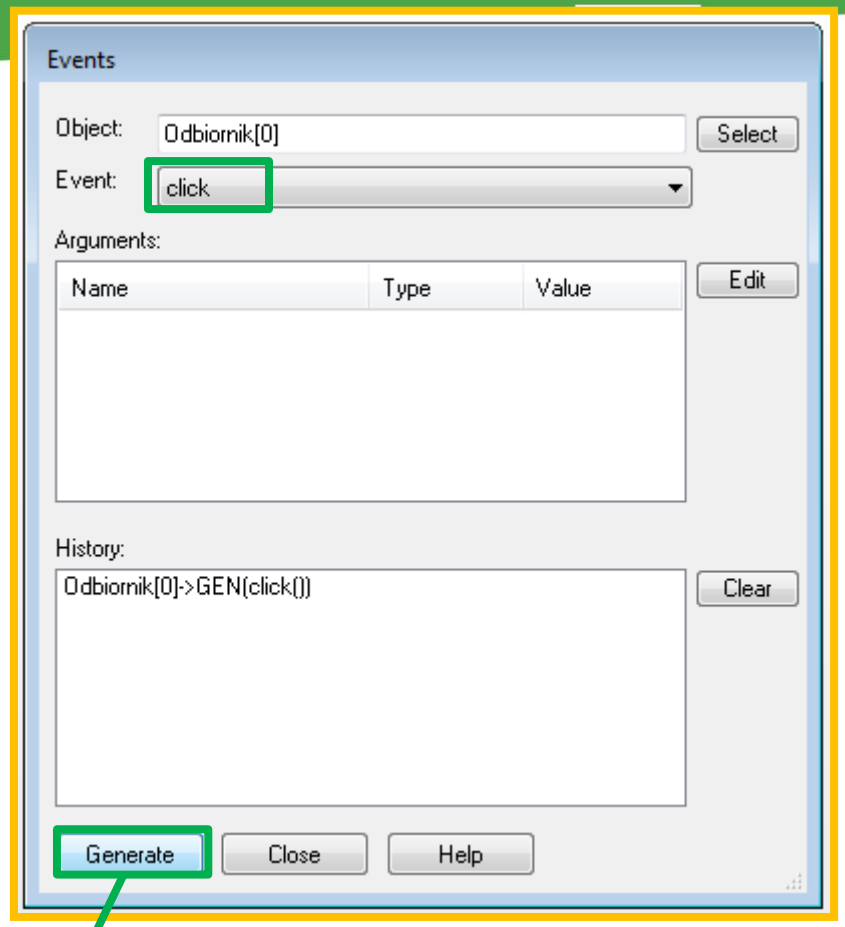
Po uruchomieniu programu tworzony jest obiekt klasy Odbiornik. Ze względu na określoną relację na Diagramie Klas równocześnie tworzony jest obiekt klasy BramaWjaz.

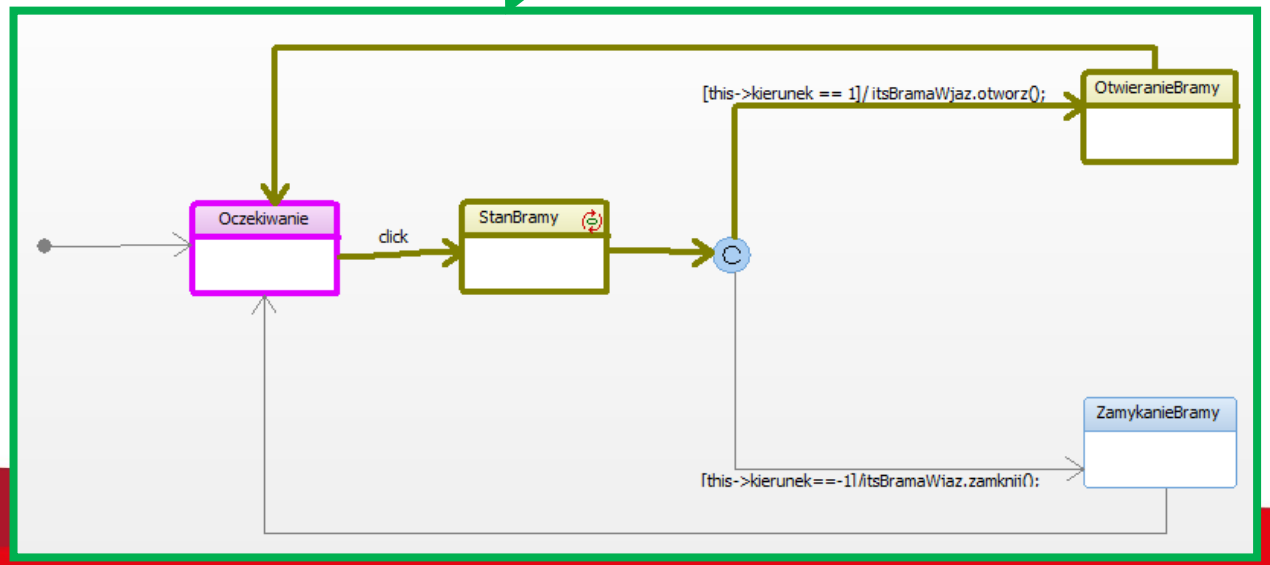
Po utworzeniu obiektów obiekt klasy Odbiornik znajduje się w stanie *Oczekiwanie*.

Testowanie

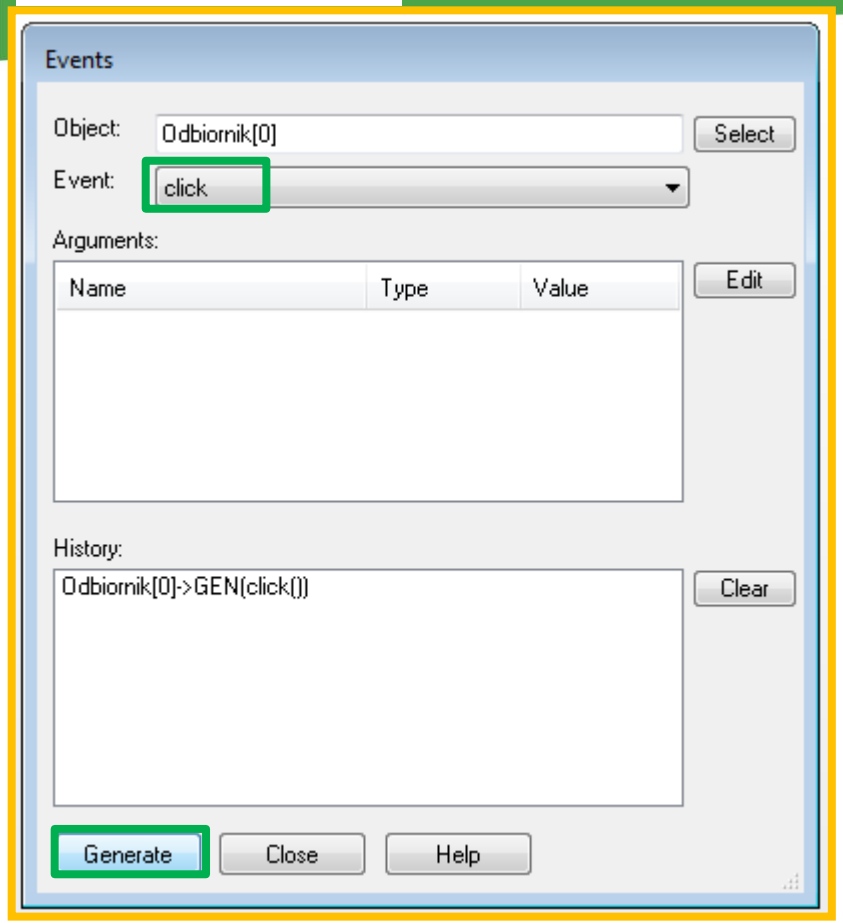

Advanced

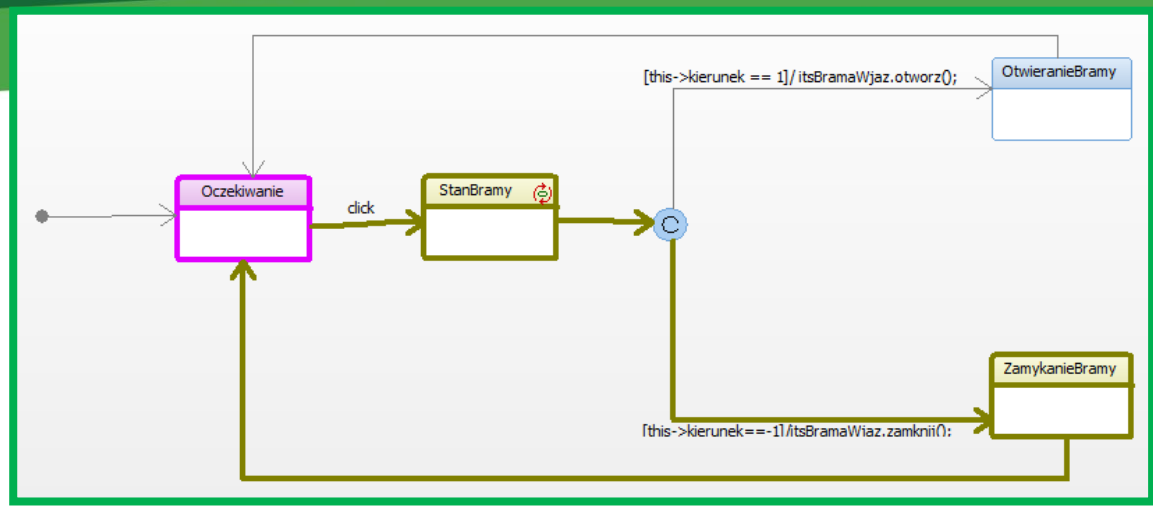

Events
Object:
Select
Instances Selection
Instance
Class
Odbiornik[0]->itsBramaWjaz
BramaWjaz
Odbiornik[0]
Odbiornik
OK
Cancel
Help
Generate
Close
Help

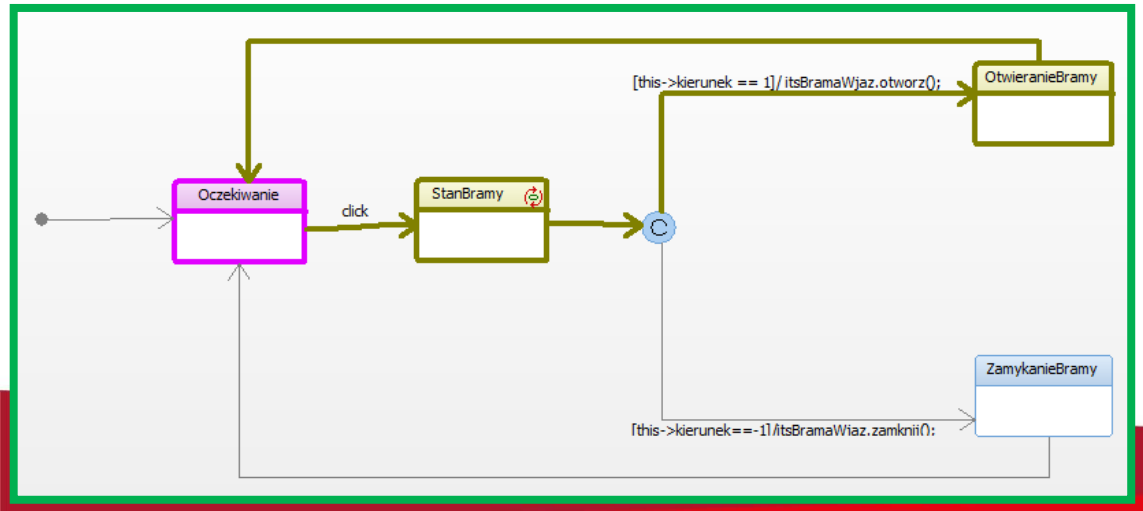

Events
Object:
Odbiornik[0]
Select
Event:
click
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->GEN(click())
Clear
Generate
Close
Help

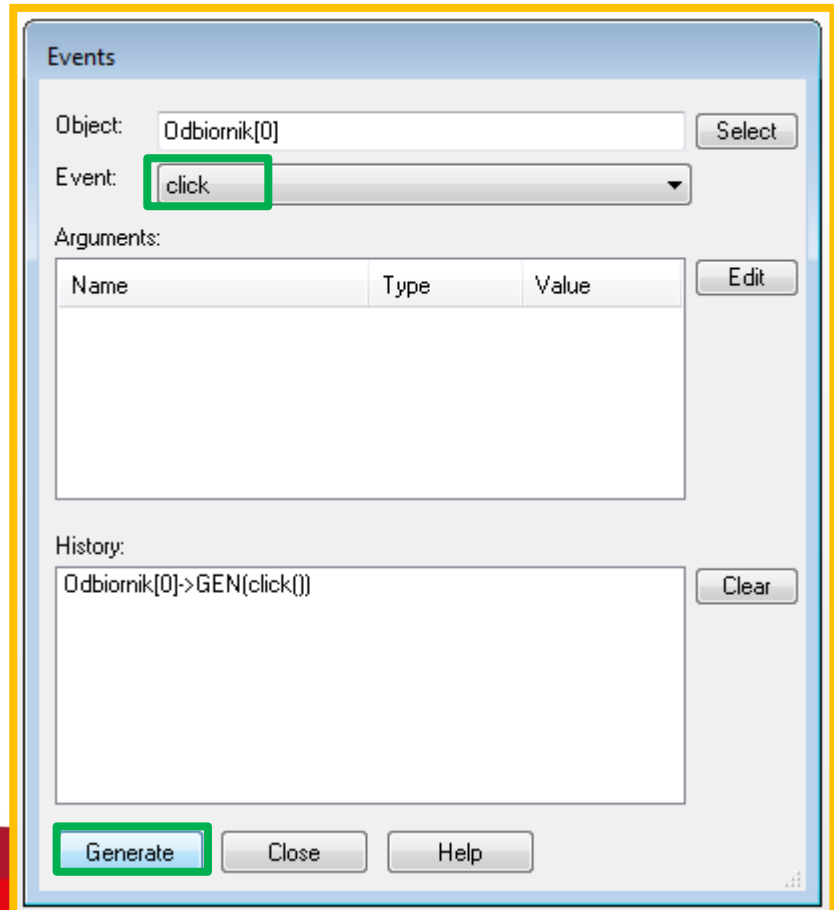

Oczekiwanie
click
StanBramy
[this->kierunek == 1]/ itsBramaWjaz.otworz();
OtwieranieBramy
ZamykanieBramy
[this->kierunek==-1]/itsBramaWjaz.zamknij();

Testowanie


Events
Object:
Odbiornik[0]
Select
Event:
click
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->GEN(click())
Clear
Generate
Close
Help


[this->kierunek == 1]/ itsBramaWjaz.otworz();
OtwieranieBramy
Oczekiwanie
click
StanBramy
ZamykanieBramy
[this->kierunek==-1]/itsBramaWiaz.zamknij();

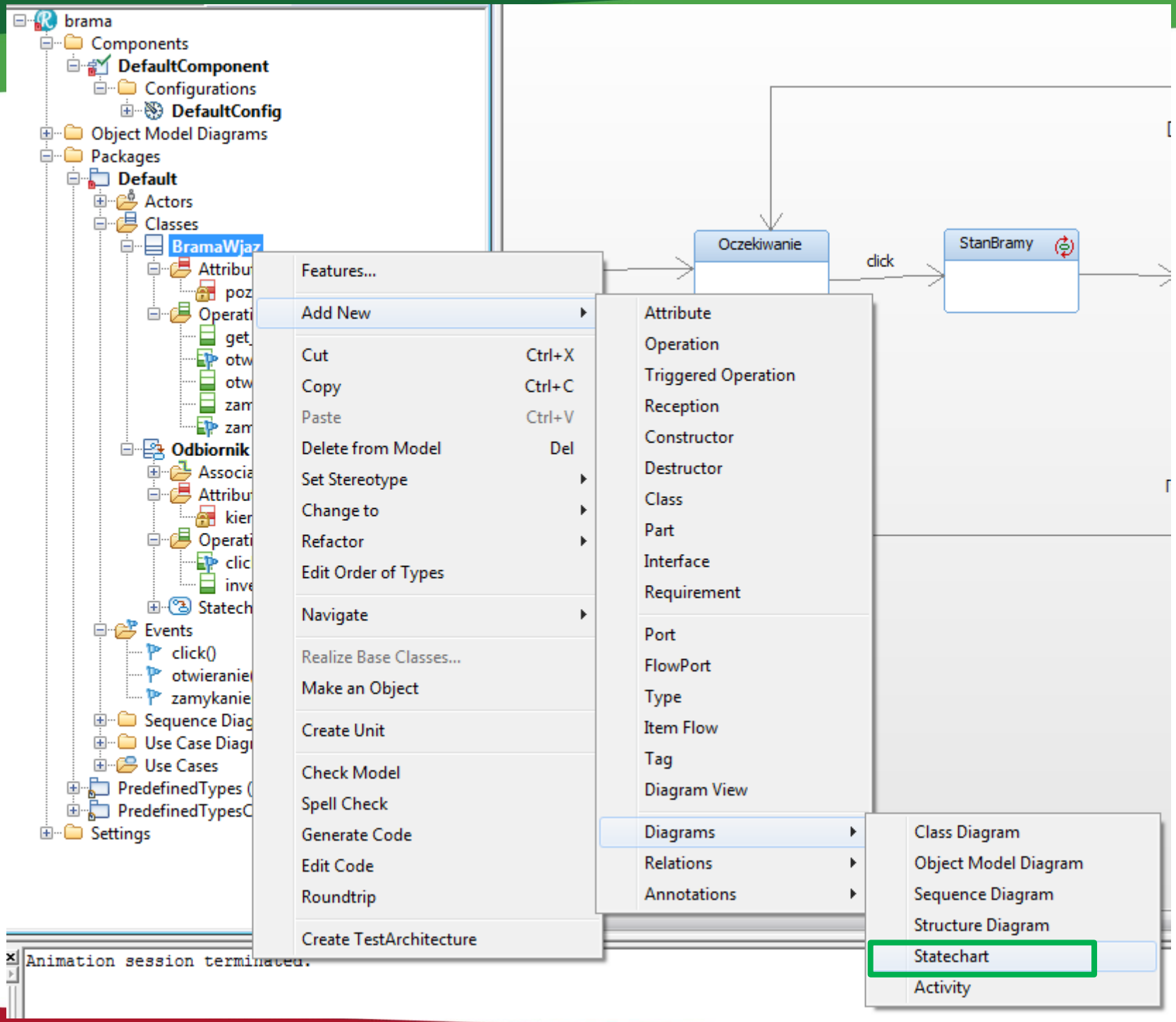

[this->kierunek == 1]/ itsBramaWjaz.otworz();
OtwieranieBramy
Oczekiwanie
click
StanBramy
ZamykanieBramy
[this->kierunek==-1]/itsBramaWiaz.zamknij();


Events
Object:
Odbiornik[0]
Select
Event:
click
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->GEN(click())
Clear
Generate
Close
Help

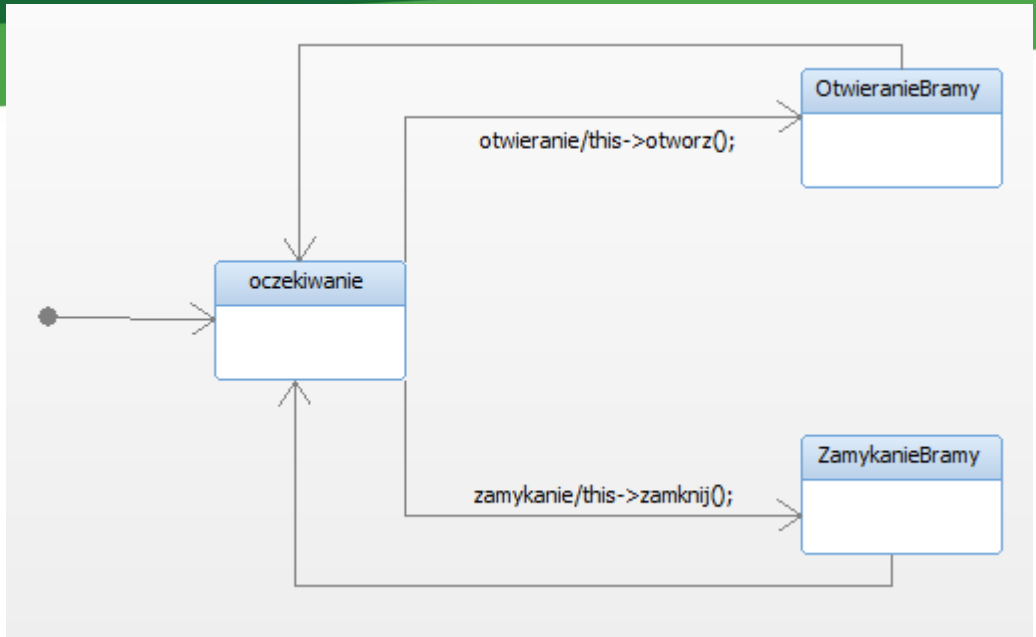
Najprostszy przykład

drugi diagram maszyny stanu

Testowanie


brama
Components
DefaultComponent
Configurations
DefaultConfig
Object Model Diagrams
Packages
Default
Actors
Classes
BramaWjaz
Odbiornik
Events
click()
Sequence Diag
Use Case Diag
Use Cases
Settings
Features...
Add New
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Delete from Model Del
Set Stereotype
Change to
Refactor
Edit Order of Types
Navigate
Realize Base Classes...
Make an Object
Create Unit
Check Model
Spell Check
Generate Code
Edit Code
Roundtrip
Create TestArchitecture
Attribute
Operation
Triggered Operation
Reception
Constructor
Destructor
Class
Part
Interface
Requirement
Port
FlowPort
Type
Item Flow
Tag
Diagram View
Diagrams
Relations
Annotations
Class Diagram
Object Model Diagram
Sequence Diagram
Structure Diagram
Statechart
Activity
Oczekiwanie
click
StanBramy
Animation session terminated.

Testowanie



Operation : otworz in BramaWjaz

General | Description | Implementation | Arguments | Relations | Tags | Properties

void otworz()

```
if (this->pozycja == 1)
{

    std::cout << "Brama jest juz otwarta!\n";

} else {

    std::cout << "Otworzono brame!\n";
    this->pozycja = 1;

}
```

Locate OK Apply

Operation : zamknij in BramaWjaz

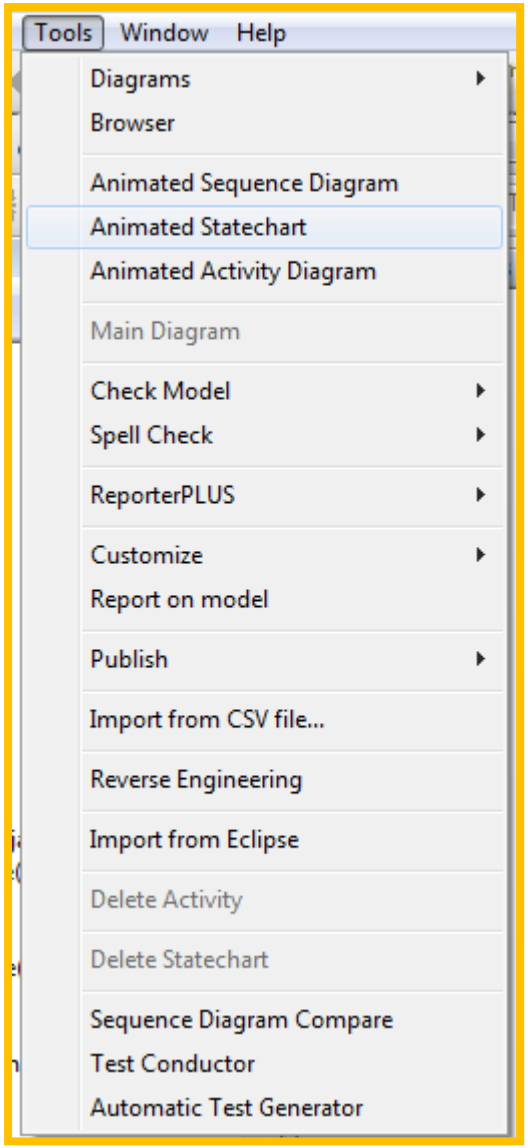
General | Description | Implementation | Arguments | Relations | Tags | Properties

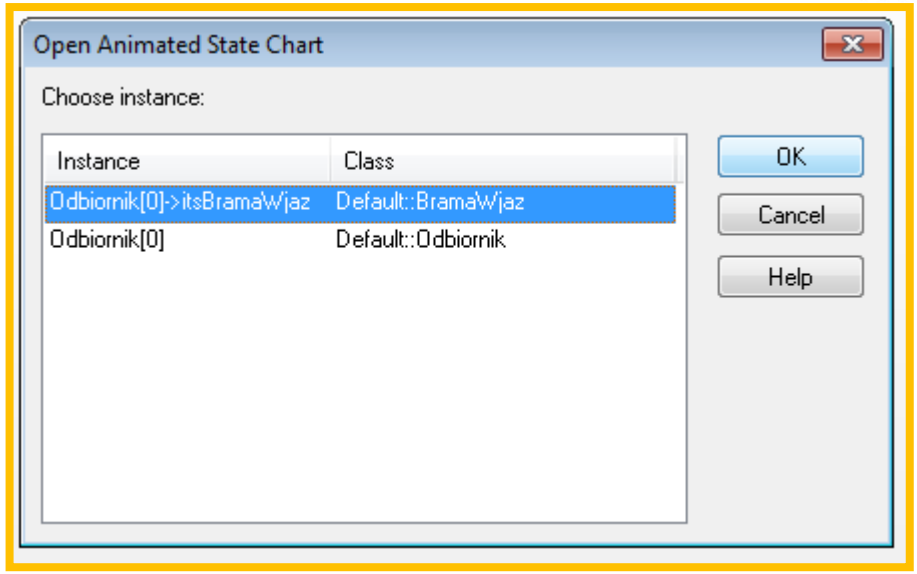
void zamknij()

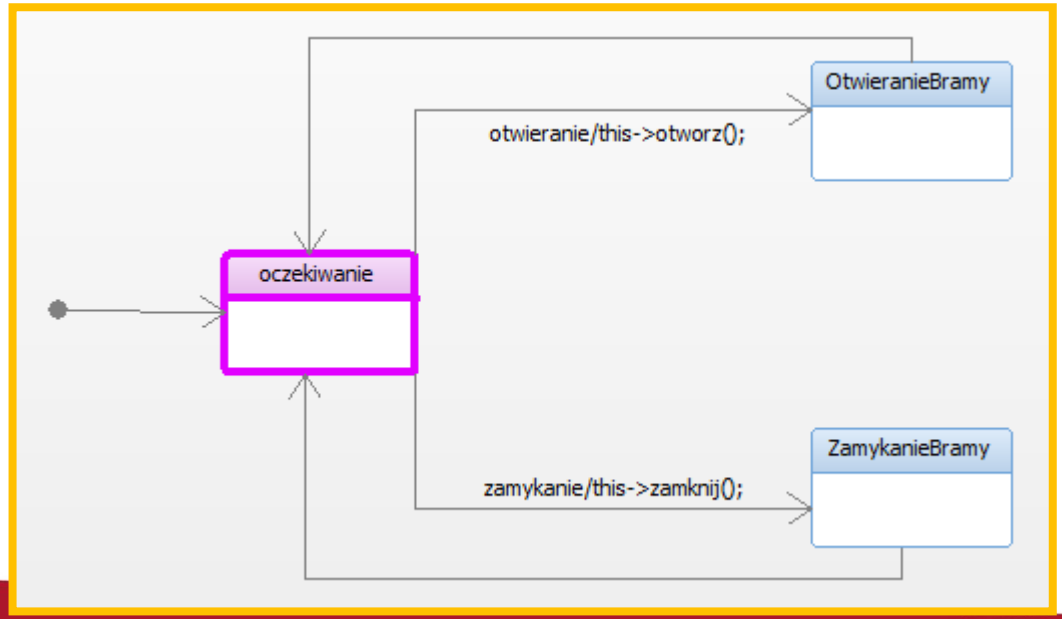
```
if (this->pozycja == -1)
{

    std::cout << "Brama jest juz zamknieta!\n";

} else {

    std::cout << "Zamknieto brame!\n";
    this->pozycja = -1;

}
```

Locate OK Apply

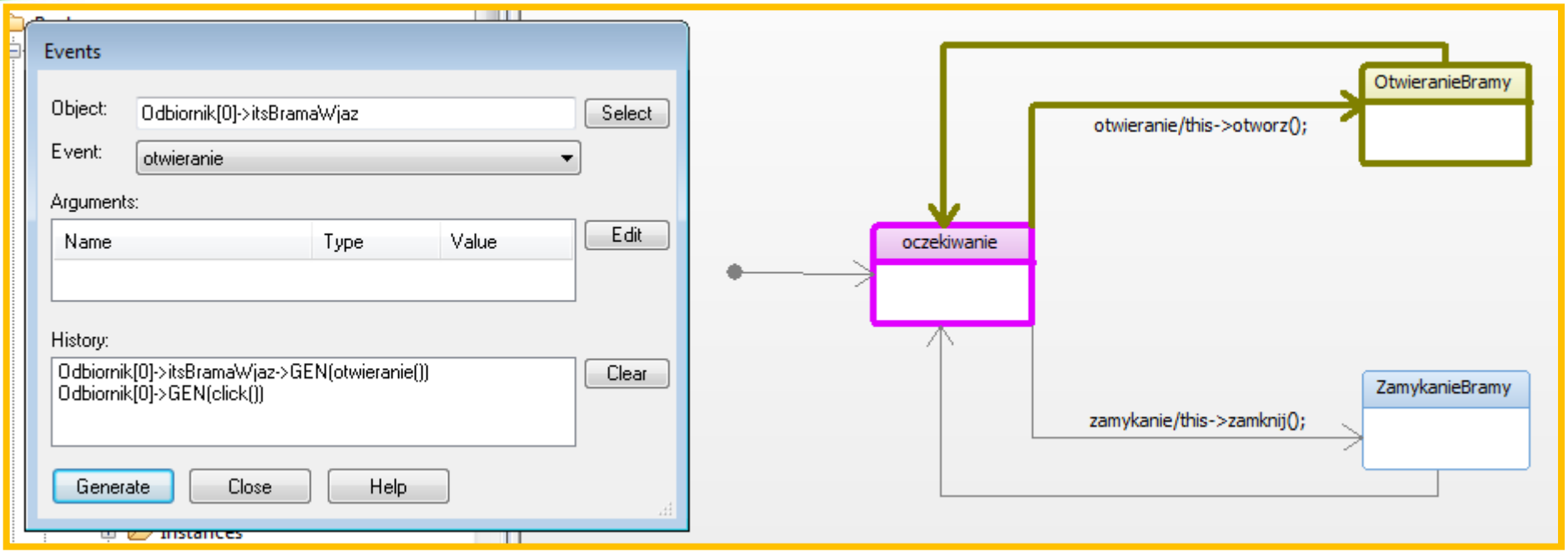
Testowanie

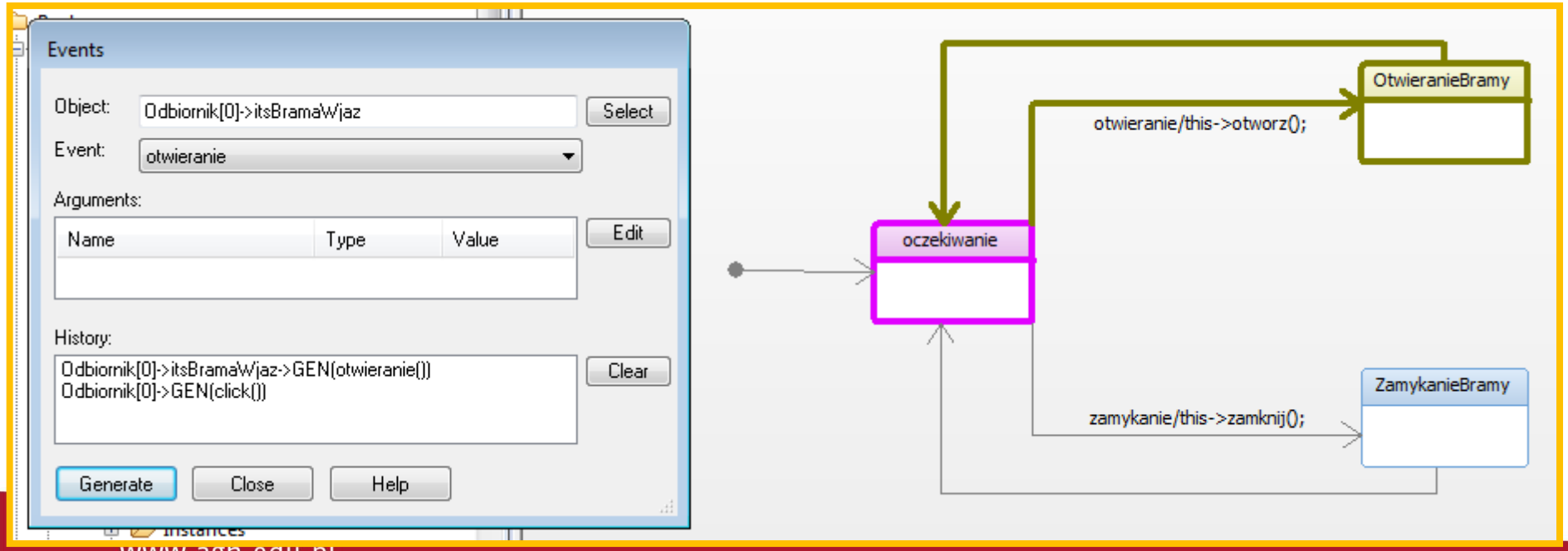

Tools
Window
Help
Diagrams
Browser
Animated Sequence Diagram
Animated Statechart
Animated Activity Diagram
Main Diagram
Check Model
Spell Check
ReporterPLUS
Customize
Report on model
Publish
Import from CSV file...
Reverse Engineering
Import from Eclipse
Delete Activity
Delete Statechart
Sequence Diagram Compare
Test Conductor
Automatic Test Generator


Open Animated State Chart
Choose instance:
Instance
Class
Odbiornik[0]->itsBramaWjaz
Default::BramaWjaz
Odbiornik[0]
Default::Odbiornik
OK
Cancel
Help

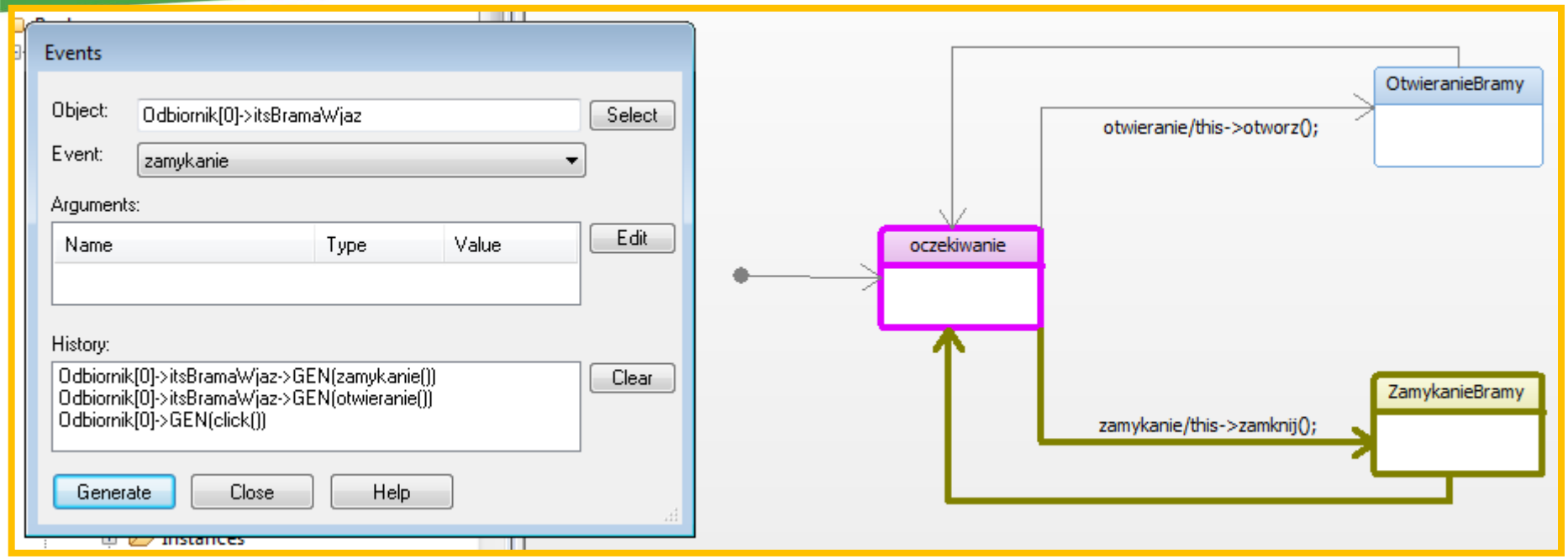

OtwieranieBramy
otwieranie/this->otworz();
oczekiwanie
ZamykanieBramy
zamykanie/this->zamknij();

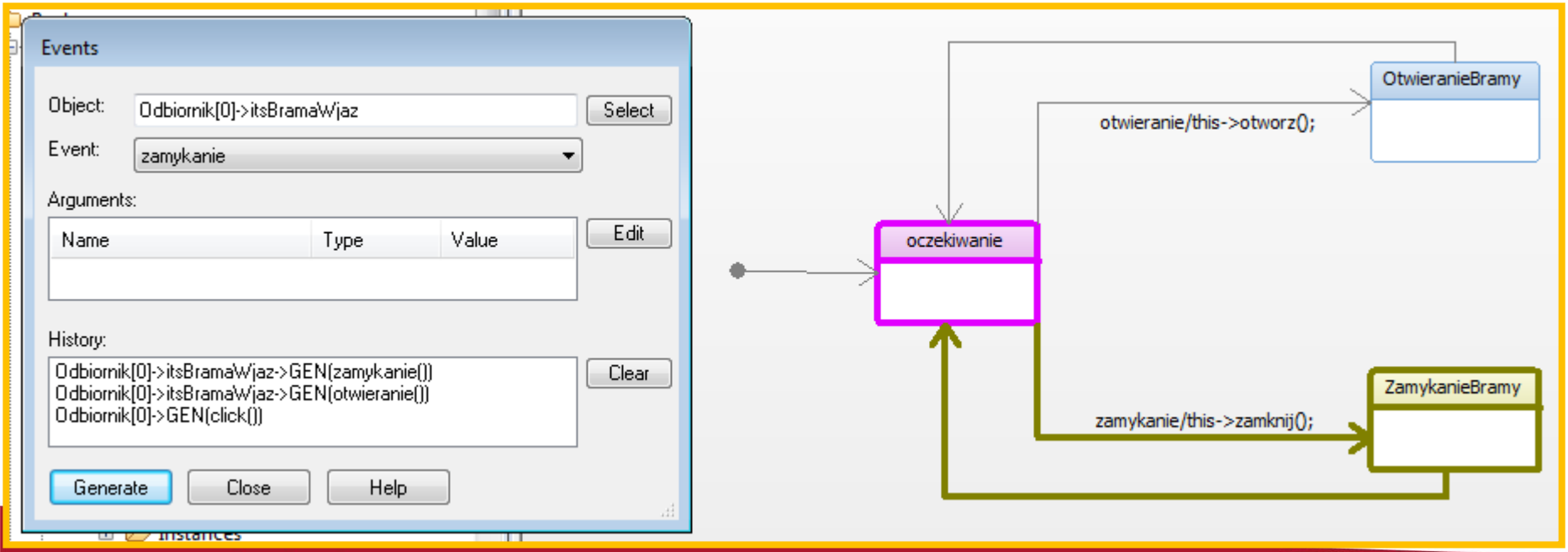
Testowanie


Events
Object:
Odbiornik[0]->itsBramaWjaz
Select
Event:
otwieranie
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->itsBramaWjaz->GEN(otwieranie())
Odbiornik[0]->GEN(click())
Clear
Generate
Close
Help
OtwieranieBramy
otwieranie/this->otworz();
oczekiwanie
ZamykanieBramy
zamykanie/this->zamknij();

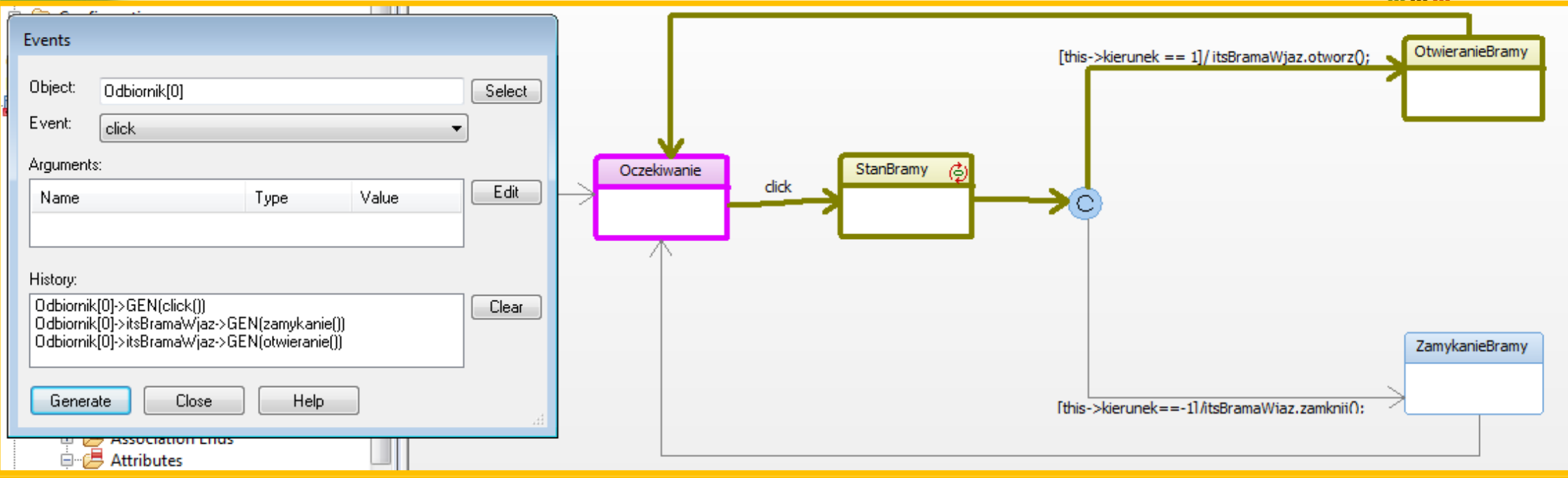

Events
Object:
Odbiornik[0]->itsBramaWjaz
Select
Event:
otwieranie
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->itsBramaWjaz->GEN(otwieranie())
Odbiornik[0]->GEN(click())
Clear
Generate
Close
Help
OtwieranieBramy
otwieranie/this->otworz();
oczekiwanie
ZamykanieBramy
zamykanie/this->zamknij();

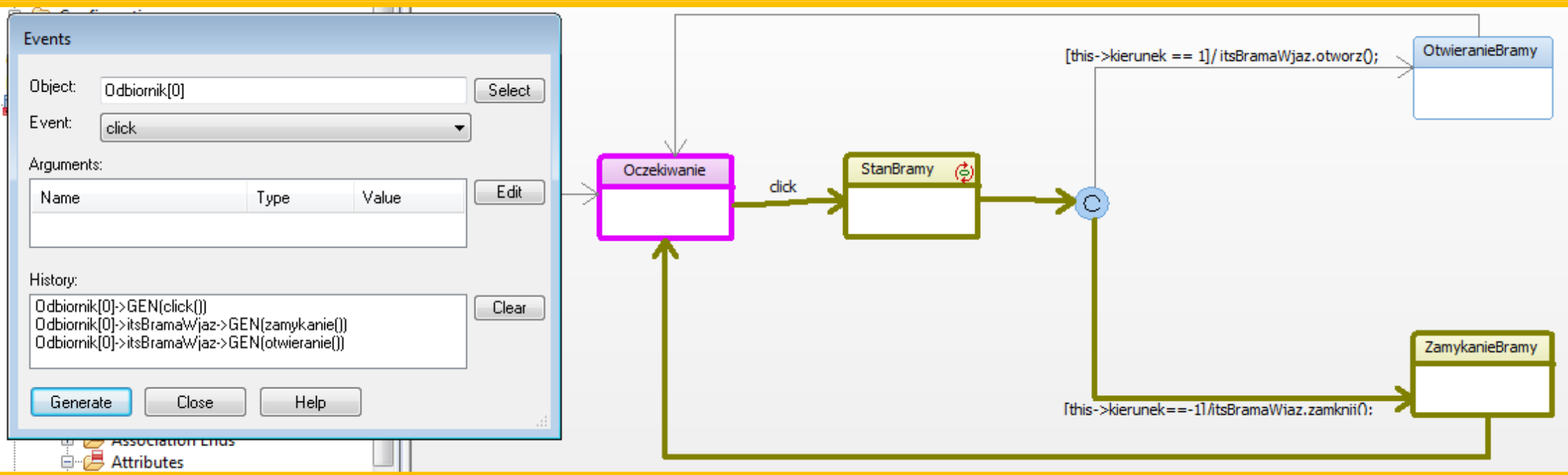
Testowanie


Events
Object:
Odbiornik[0]->itsBramaWjaz
Select
Event:
zamykanie
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->itsBramaWjaz->GEN(zamykanie())
Odbiornik[0]->itsBramaWjaz->GEN(otwieranie())
Odbiornik[0]->GEN(click())
Clear
Generate
Close
Help
OtwieranieBramy
otwieranie/this->otworz();
oczekiwanie
ZamykanieBramy
zamykanie/this->zamknij();

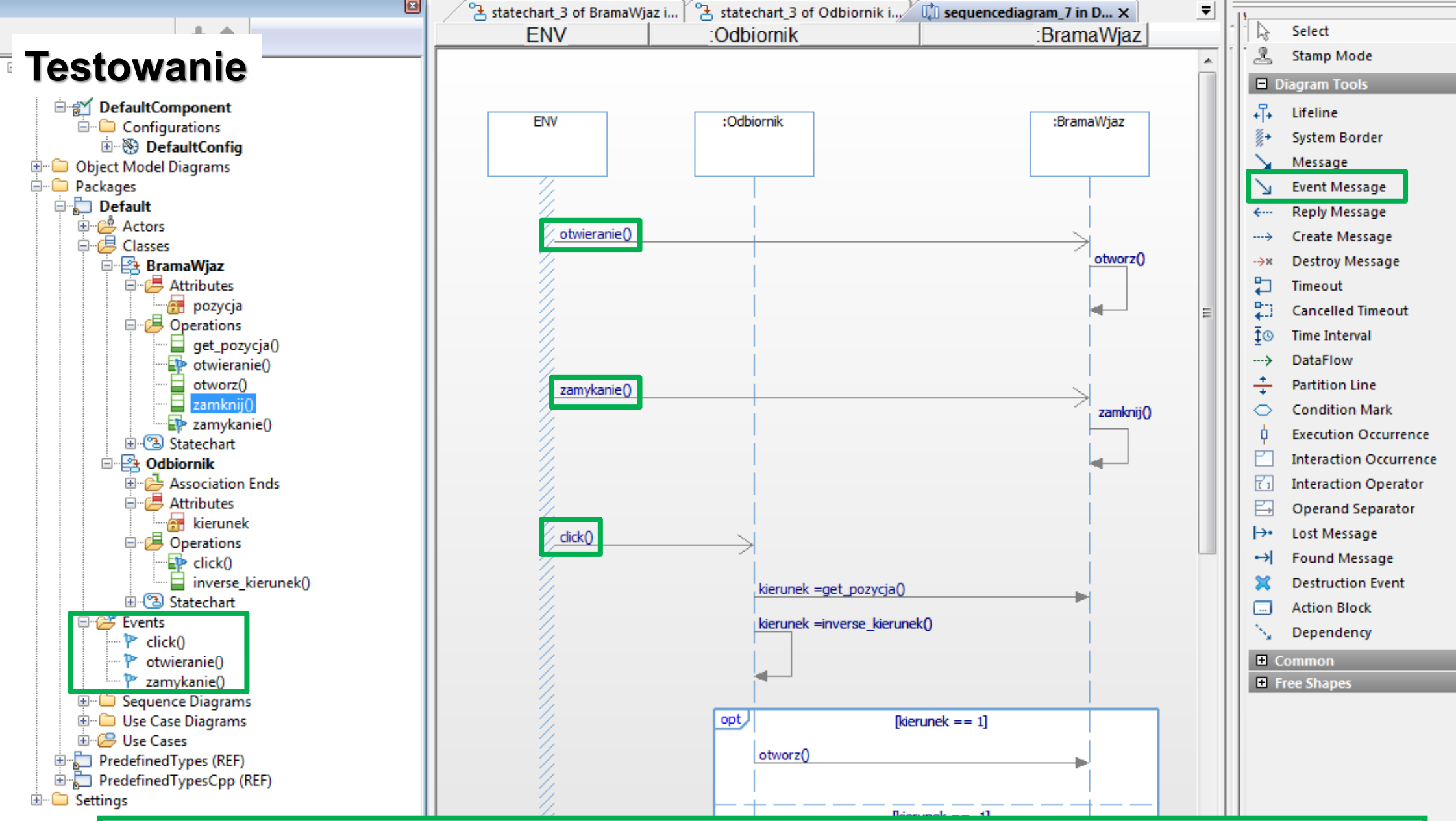

Events
Object:
Odbiornik[0]->itsBramaWjaz
Select
Event:
zamykanie
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->itsBramaWjaz->GEN(zamykanie())
Odbiornik[0]->itsBramaWjaz->GEN(otwieranie())
Odbiornik[0]->GEN(click())
Clear
Generate
Close
Help
OtwieranieBramy
otwieranie/this->otworz();
oczekiwanie
ZamykanieBramy
zamykanie/this->zamknij();

Testowanie


Events
Object:
Odbiornik[0]
Select
Event:
click
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->GEN(click())
Odbiornik[0]->itsBramaWjaz->GEN(zamykanie())
Odbiornik[0]->itsBramaWjaz->GEN(otwieranie())
Clear
Generate
Close
Help
Attributes
Oczekiwanie
click
StanBramy
[this->kierunek == 1]/ itsBramaWjaz.otworz();
OtwieranieBramy
ZamykanieBramy
[this->kierunek==-1]/itsBramaWjaz.zamknij();

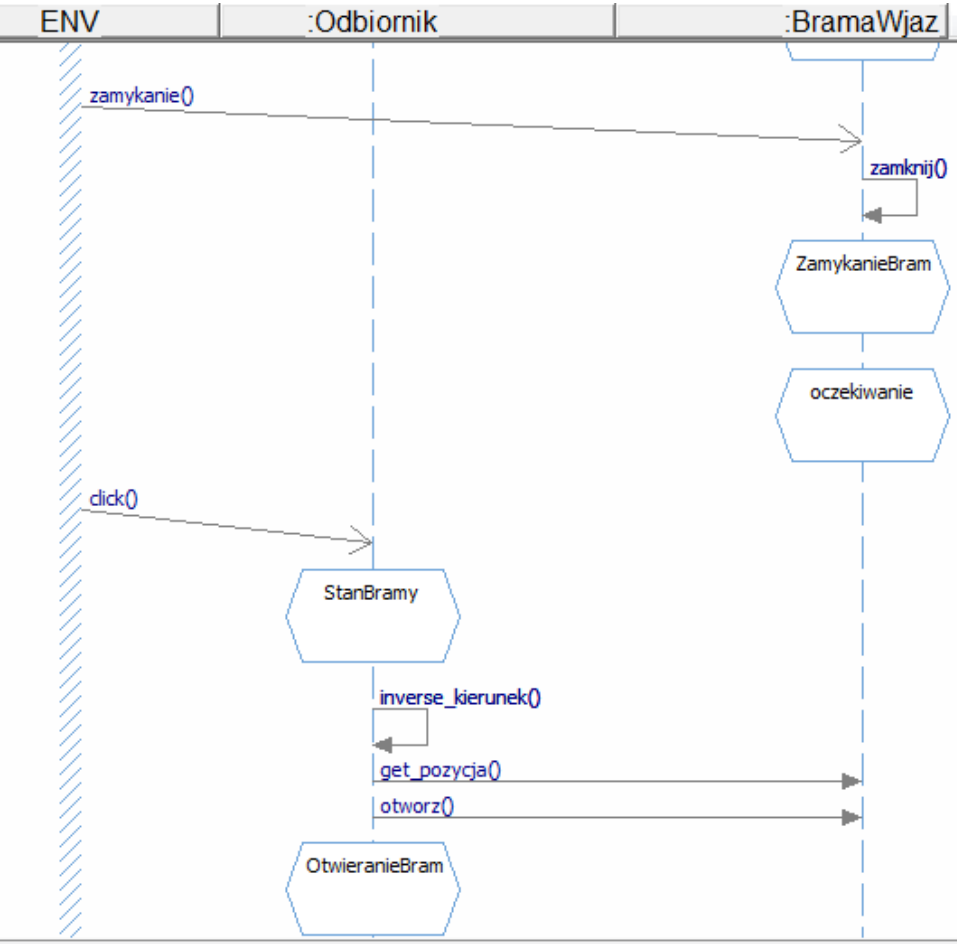

Events
Object:
Odbiornik[0]
Select
Event:
click
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->GEN(click())
Odbiornik[0]->itsBramaWjaz->GEN(zamykanie())
Odbiornik[0]->itsBramaWjaz->GEN(otwieranie())
Clear
Generate
Close
Help
Attributes
Oczekiwanie
click
StanBramy
[this->kierunek == 1]/ itsBramaWjaz.otworz();
OtwieranieBramy
ZamykanieBramy
[this->kierunek==-1]/itsBramaWjaz.zamknij();

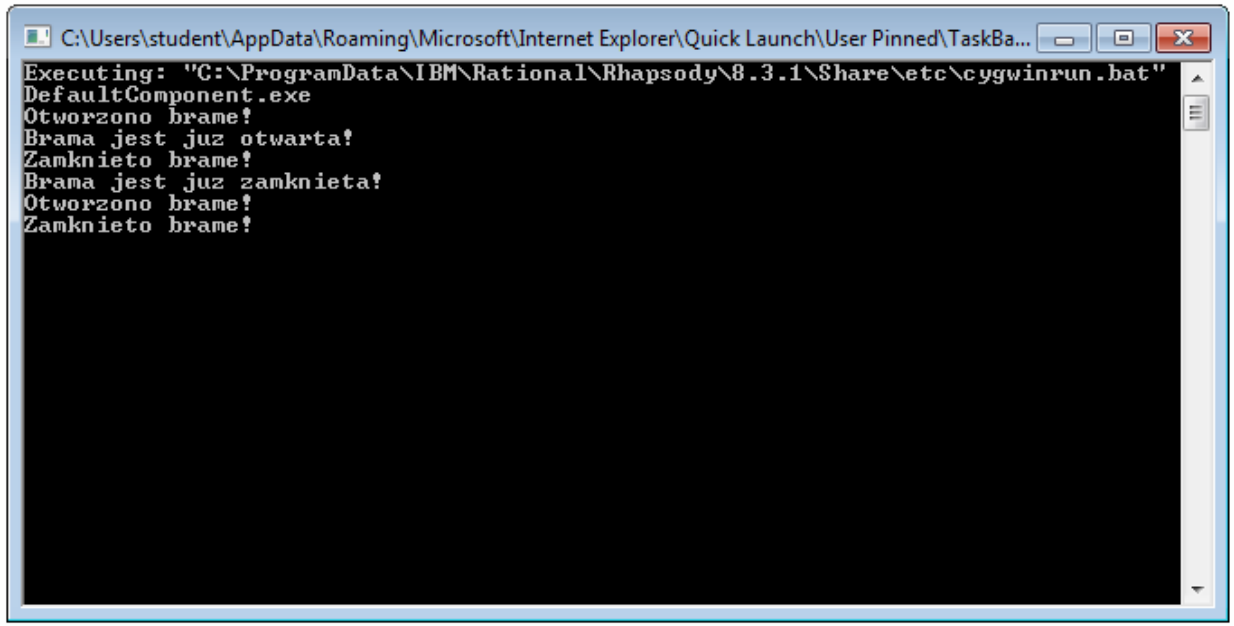
Testowanie



Jak można zauważyć na omawianym przykładzie, przepływem sterowania w trakcie testu można sterować za pomocą wymuszeń [Triggerów] zdefiniowanych na diagramie sekwencji jako zdarzenia typu Event Message – sygnały asynchroniczne.

Testowanie

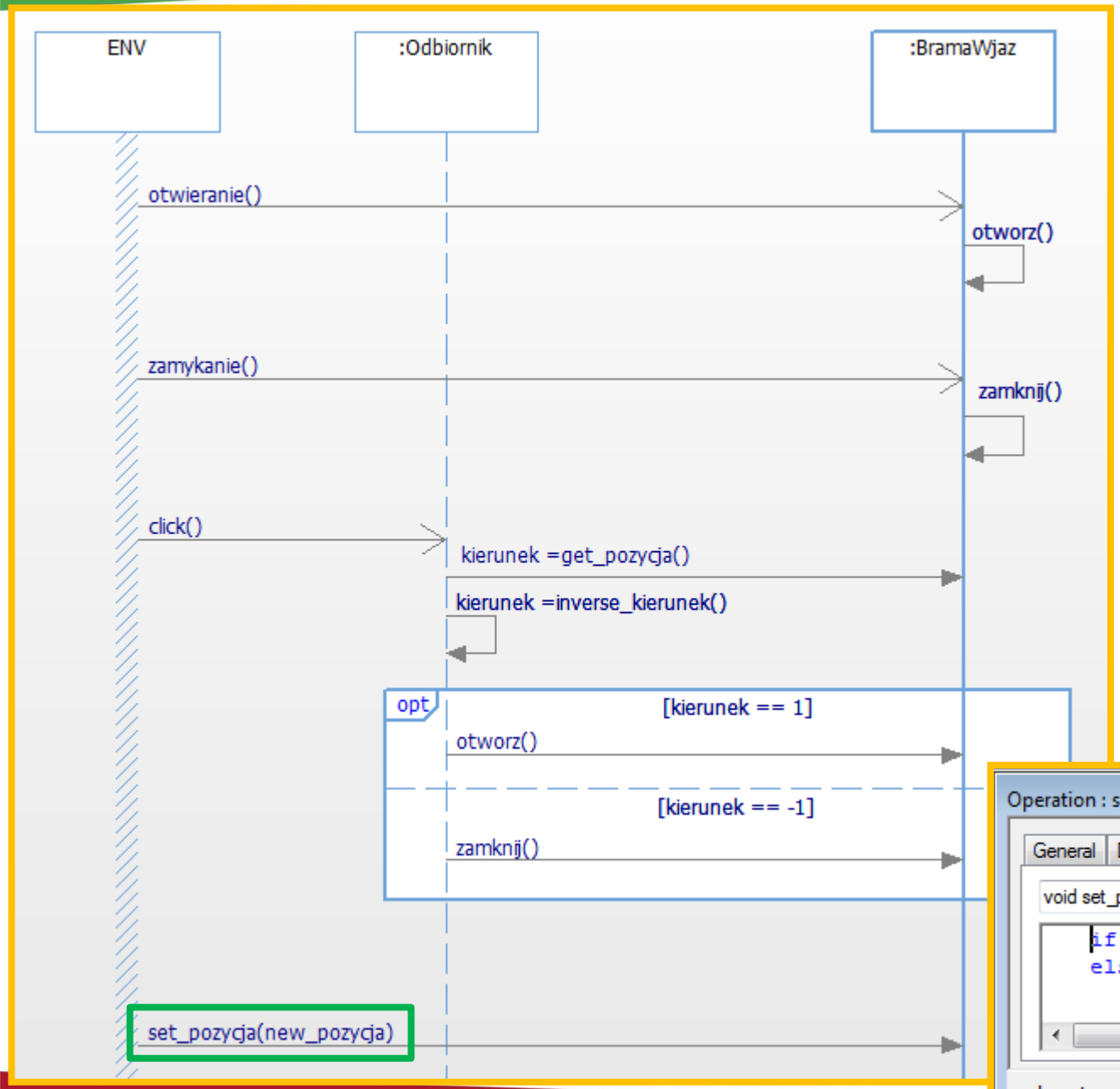

ENV
:Odbiornik
:BramaWjaz
zamykanie()
zamknij()
ZamykanieBram
oczekiwanie
click()
StanBramy
inverse_kierunek()
get_pozycja()
otworz()
OtwieranieBram


C:\Users\student\AppData\Roaming\Microsoft\Internet Explorer\Quick Launch\User Pinned\TaskBa...
Executing: "C:\ProgramData\IBM\Rational\Rhapsody\8.3.1\Share\etc\cygwinrun.bat"
DefaultComponent.exe
Otworzono brame!
Brama jest juz otwarta!
Zamknieto brame!
Brama jest juz zamknieta!
Otworzono brame!
Zamknieto brame!

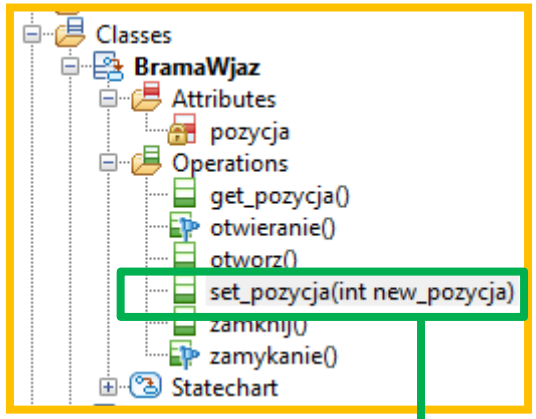
Najprostszy przykład

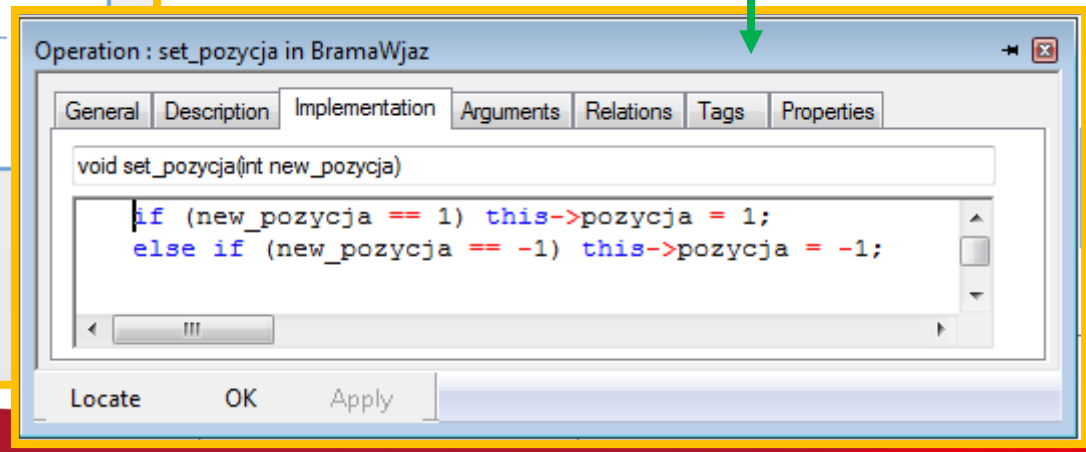
wywołanie funkcji w trakcie testu

Testowanie

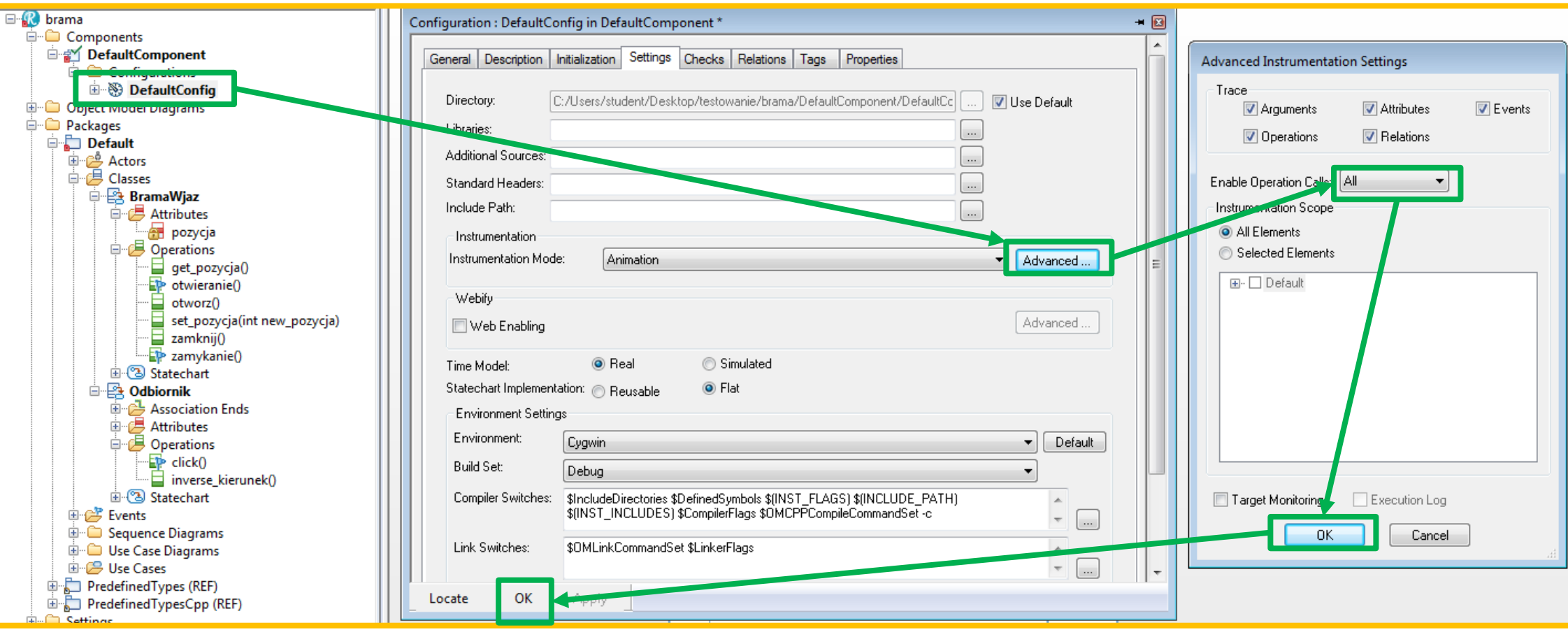


Dodajemy nową funkcję składową klasy BramaWjaz, która pozwoli na zmianę stanu bramy, tj. bezpośrednią zmianę wartości parametru składowego.

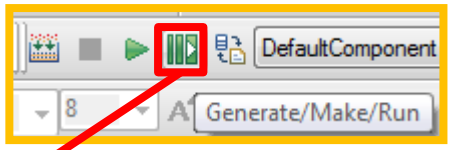


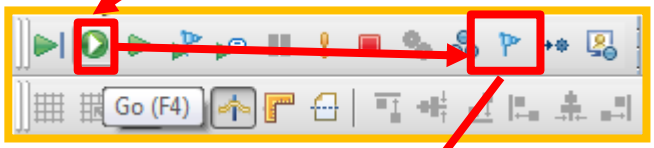


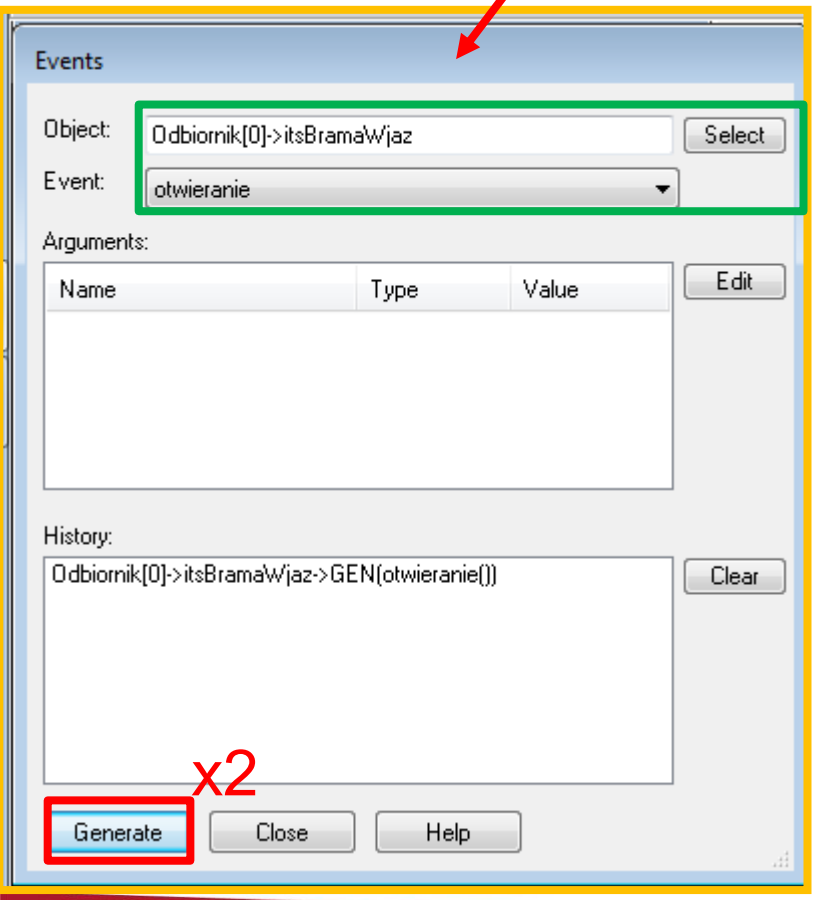
Testowanie

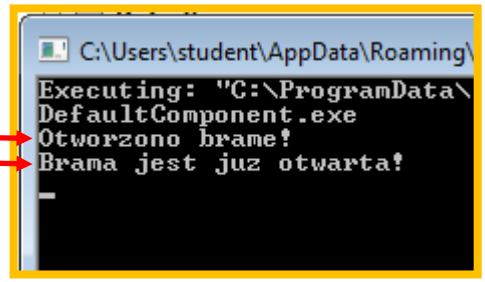

brama
Components
DefaultComponent
DefaultConfig
Packages
Default
Actors
Classes
BramaWjaz
Attributes
pozycja
Operations
get_pozycja()
otwieranie()
otworz()
set_pozycja(int new_pozycja)
zamknij()
zamykanie()
Statechart
Odbiornik
Association Ends
Attributes
Operations
click()
inverse_kierunek()
Statechart
Events
Sequence Diagrams
Use Case Diagrams
Use Cases
PredefinedTypes (REF)
PredefinedTypesCpp (REF)
Configuration : DefaultConfig in DefaultComponent *
General
Description
Initialization
Settings
Checks
Relations
Tags
Properties
Directory:
C:/Users/student/Desktop/testowanie/brama/DefaultComponent/DefaultCc
Use Default
Libraries:
Additional Sources:
Standard Headers:
Include Path:
Instrumentation
Instrumentation Mode:
Animation
Advanced ...
Webify
Web Enabling
Advanced ...
Time Model:
Real
Simulated
Statechart Implementation:
Reusable
Flat
Environment Settings
Environment:
Cygwin
Default
Build Set:
Debug
Compiler Switches:
$IncludeDirectories $DefinedSymbols $(INST_FLAGS) $(INCLUDE_PATH)
$(INST_INCLUDES) $CompilerFlags $OMCPPCompileCommandSet -c
Link Switches:
$OMLinkCommandSet $LinkerFlags
Locate
OK
Advanced Instrumentation Settings
Trace
Arguments
Attributes
Events
Operations
Relations
Enable Operation Calls:
All
Instrumentation Scope
All Elements
Selected Elements
Default
Target Monitoring
Execution Log
OK
Cancel

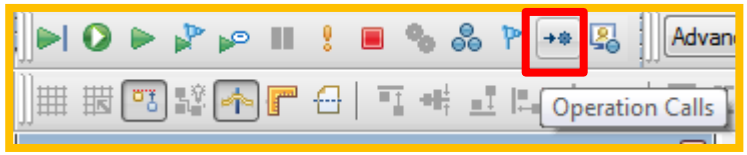
Testowanie


DefaultComponent
Generate/Make/Run


Go (F4)

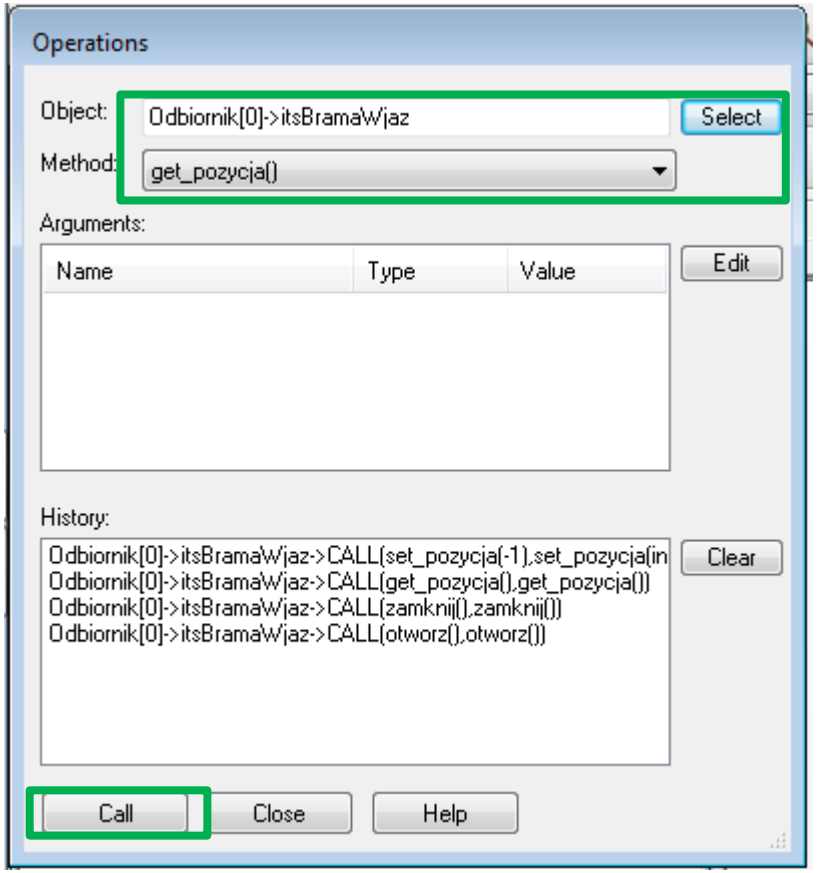

Events
Object:
Odbiornik[0]->itsBramaWjaz
Select
Event:
otwieranie
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->itsBramaWjaz->GEN(otwieranie())
Clear
x2
Generate
Close
Help


C:\Users\student\AppData\Roaming\
Executing: "C:\ProgramData\
DefaultComponent.exe
Otworzono brame!
Brama jest juz otwarta!

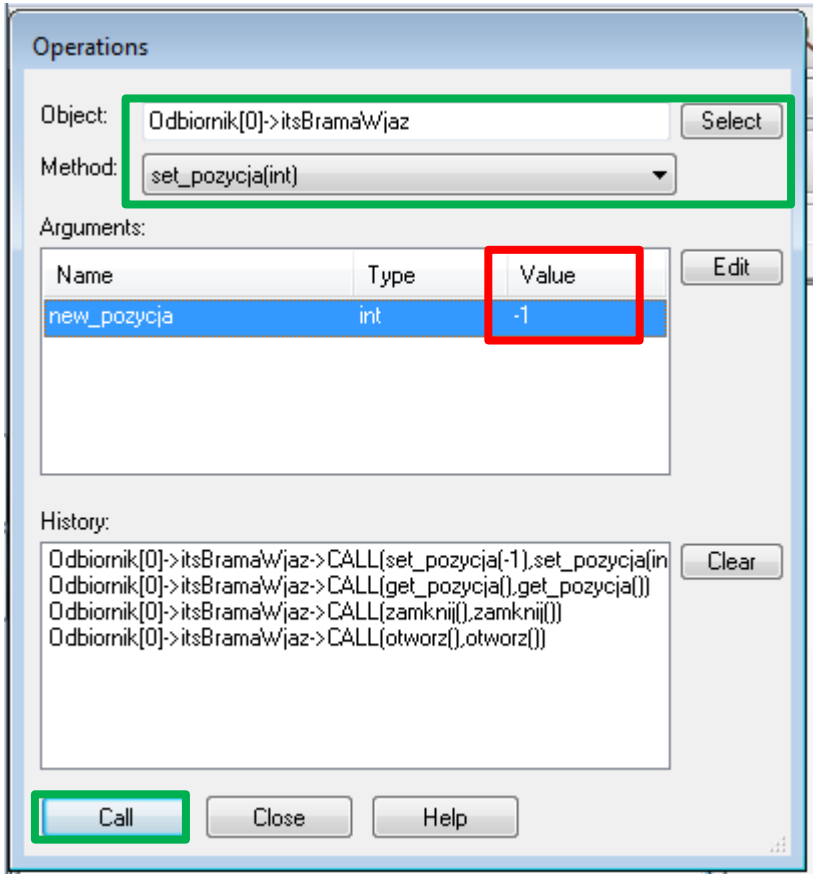

Advan
Operation Calls

Testowanie

1

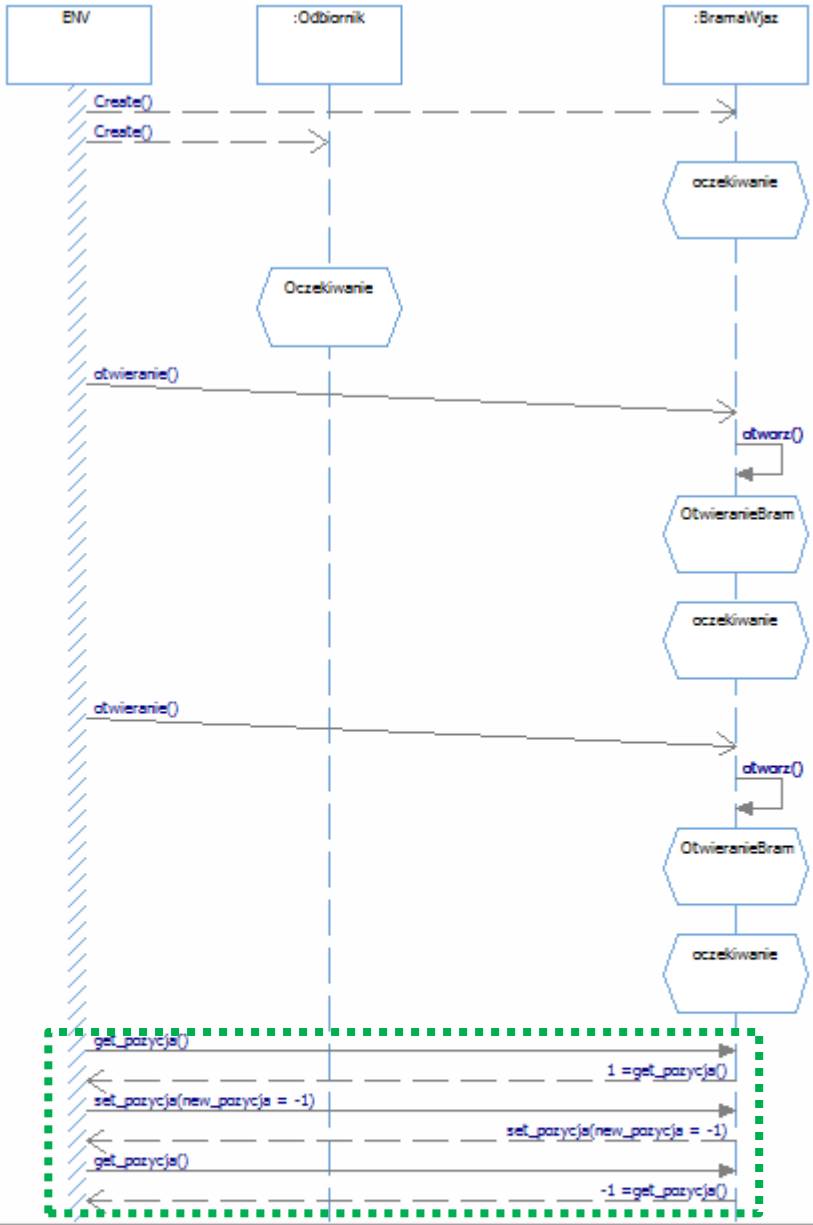


2



3

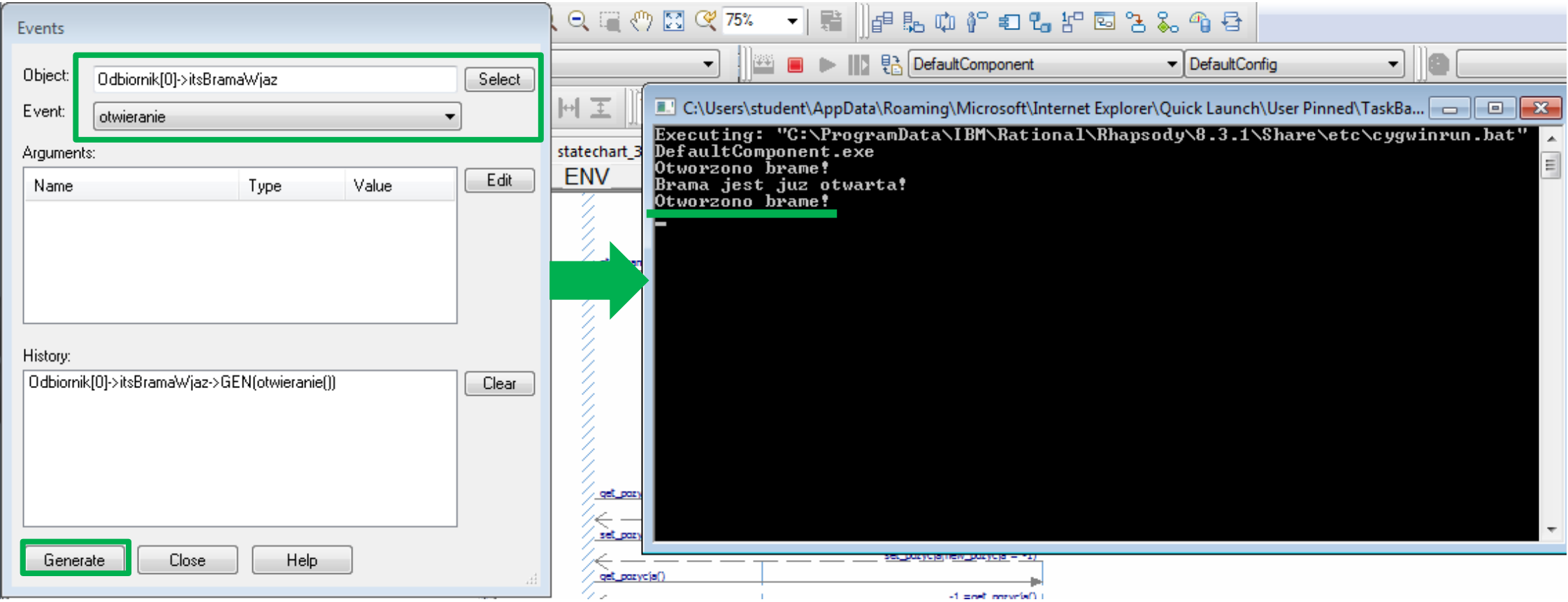
Ponowne pobranie informacji o pozycji bramy



```
Message: Odbiornik[0]->itsBramaWjaz->CALL(get_pozycja(),get_pozycja()) sent.
Message: Odbiornik[0]->itsBramaWjaz->CALL(set_pozycja(-1),set_pozycja(int)) sent.
Message: Odbiornik[0]->itsBramaWjaz->CALL(get_pozycja(),get_pozycja()) sent.
Odbiornik[0]->itsBramaWjaz->CALL(get_pozycja(),get_pozycja()) returned 1.
Odbiornik[0]->itsBramaWjaz->CALL(set_pozycja(-1),set_pozycja(int)) returned .
Odbiornik[0]->itsBramaWjaz->CALL(get_pozycja(),get_pozycja()) returned -1.
```

t Queue | Log | Check Model | Build | Configuration Management | Animation

Testowanie


Events
Object:
Odbiornik[0]->itsBramaWjaz
Select
Event:
otwieranie
Arguments:
Name
Type
Value
Edit
History:
Odbiornik[0]->itsBramaWjaz->GEN(otwieranie())
Clear
Generate
Close
Help
75%
DefaultComponent
DefaultConfig
statechart_3
ENV
C:\Users\student\AppData\Roaming\Microsoft\Internet Explorer\Quick Launch\User Pinned\TaskBa...
Executing: "C:\ProgramData\IBM\Rational\Rhapsody\8.3.1\Share\etc\cygwinrun.bat"
DefaultComponent.exe
Otworzono brame!
Brama jest juz otwarta!
Otworzono brame!